

DEEP LEARNING

Note : Images are not own, obtained from google through various sources



LIST OF IMPORTANT DEEP LEARNING TOPICS

All Rights Reserved. Author @ Rajendra Phani

- Introduction to Deep Learning & Neural Networks
- Biological vs Artificial Neurons
- Perceptron & Multi-Layer Perceptron (MLP)
- Activation Functions
- Forward Propagation
- Loss Functions
- Gradient Descent & Backpropagation
- Optimization Algorithms (SGD, Momentum, Adam)
- Overfitting & Regularization (L1/L2, Dropout, Early Stopping)
- Weight Initialization Techniques
- Batch Normalization
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs, LSTM, GRU)
- Autoencoders & Variational Autoencoders (VAEs)
- Generative Adversarial Networks (GANs)
- Attention Mechanism & Transformers
- Transfer Learning & Fine-Tuning

1. INTRODUCTION TO DEEP LEARNING & NEURAL NETWORKS

All Rights Reserved. Author @ Rajendra Phani

- What is Deep Learning?
- Subset of machine learning using *deep neural networks* (many hidden layers).
- Learns hierarchical representations directly from raw data (e.g., images, text, audio).
- Excels at tasks where feature engineering is hard: image recognition, NLP, speech.

Why “Deep”?

- Depth → Hierarchical feature learning:
 - Layer 1: edges
 - Layer 2: textures
 - Layer 3: parts (e.g., eyes, wheels)
 - Layer 4: objects (e.g., face, car)

Key Milestones

- 2012: AlexNet (ImageNet) → DL revolution
- 2014: VAEs, GANs
- 2017: Transformers

DEFINITION OF A PERCEPTRON / NEURON

All Rights Reserved. Author @ Rajendra Phani

An artificial neuron (or perceptron) is the basic computational unit of a neural network that takes some inputs, combines them numerically, and decides an output, usually for classification tasks. It is inspired by a biological neuron but is implemented as a simple mathematical function in deep learning models.

Structure of a perceptron

A perceptron has:

- Inputs: numerical features $x_1, x_2, \dots, x_n$.
- Weights: parameters $w_1, w_2, \dots, w_n$ that tell how important each input is.
- Bias: an extra parameter that shifts the decision boundary.
- Activation function: a function (often a step or sigmoid) that turns the combined input into an output (like 0/1).

Mathematically, it computes a weighted sum and then applies an activation:

$$\text{output} = f(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b)$$

This lets the perceptron decide whether to “fire” (e.g., output 1) or not (output 0).

Role in deep learning

A single perceptron is a linear classifier, mainly useful for simple, linearly separable problems. Deep learning stacks many such artificial neurons into layers (input, hidden, output), forming multilayer perceptrons and other architectures that can learn complex, non-linear patterns. In this sense, every node in a dense neural network layer is essentially an artificial neuron using this same weighted-sum-plus-activation idea

2. BIOLOGICAL VS ARTIFICIAL NEURONS

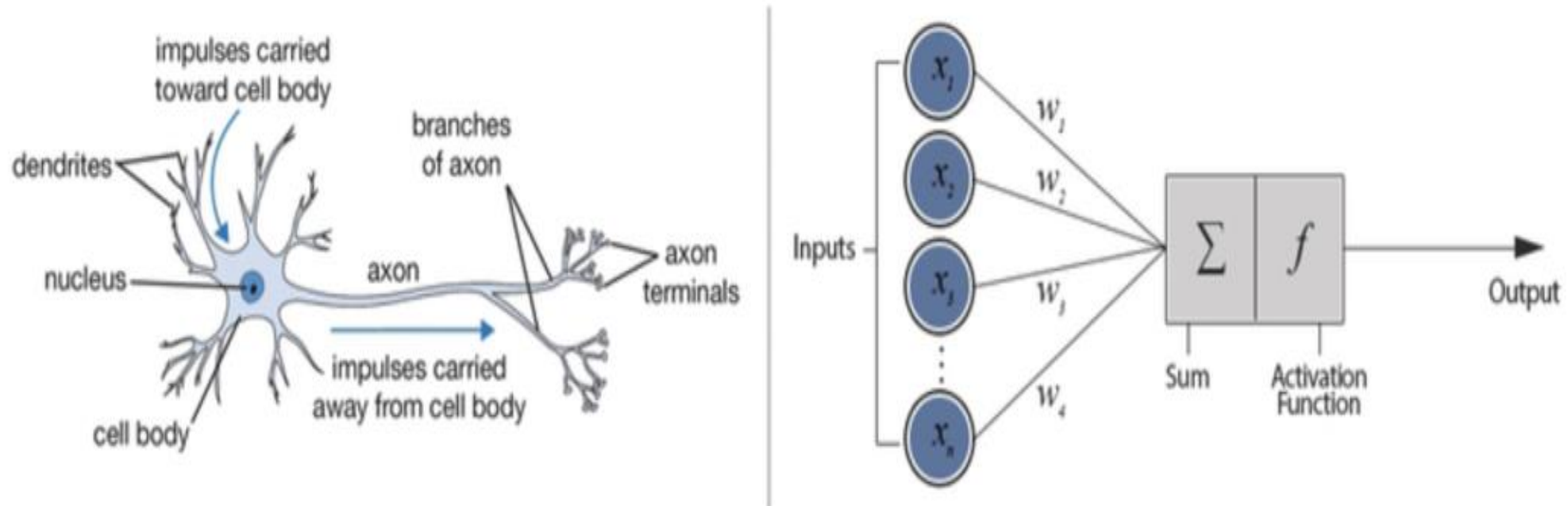
All Rights Reserved. Author @ Rajendra Phani

- Biological Neuron
- Dendrites (input), Soma (integration), Axon (output), Synapse (weight/strength)
- Artificial Neuron (McCulloch–Pitts, 1943)
- Inputs: $x_1, x_2, \dots, x_n$
- Weights: $w_1, w_2, \dots, w_n$
- Bias: b
- Output:
- $z = \sum_{i=1}^n w_i x_i + b$ $a = \sigma(z)$
- σ : activation function (e.g., sigmoid, ReLU)

BIOLOGICAL VS ARTIFICIAL NEURONS

All Rights Reserved. Author @ Rajendra Phani

Biological Neuron versus Artificial Neural Network



ARTIFICIAL NEURON CODE

All Rights Reserved. Author @ Rajendra Phani

```
import tensorflow as tf
```

```
# Single neuron:  $y = \sigma(w \cdot x + b)$ 
```

```
x = tf.constant([[2.0, 3.0]])
```

```
# shape (1, 2)
```

```
w = tf.Variable([[0.5], [0.3]])
```

```
# shape (2, 1)
```

```
b = tf.Variable(0.1)
```

```
# scalar bias
```

```
z = tf.matmul(x, w) + b
```

```
# Linear:  $z = w \cdot x + b$ 
```

```
a = tf.nn.sigmoid(z)
```

```
# Activation:  $a = \sigma(z)$ 
```

```
print("z =", z.numpy(), "→ a =", a.numpy())
```

PERCEPTRON & MULTI-LAYER PERCEPTRON (MLP)

All Rights Reserved. Author @ Rajendra Phani

What is a perceptron?

A perceptron takes several input features $x_1, x_2, \dots, x_n$, multiplies each by a weight, adds a bias, and computes a weighted sum $z = w_1x_1 + \dots + w_nx_n + b$. This sum is then passed through a simple activation function (classically a step function) to output either 0 or 1, so it acts as a linear binary classifier with a straight-line (or hyperplane) decision boundary.

Conceptually, you can think of a perceptron as answering “yes” or “no” based on whether the weighted sum of evidence crosses a threshold, which is why it only works when the two classes are linearly separable. The perceptron learning algorithm adjusts the weights and bias based on classification errors, gradually rotating and shifting that linear boundary until training examples are correctly separated (if possible).

What is a multilayer perceptron (MLP)?

A multilayer perceptron is a feedforward neural network built from multiple layers of perceptron-like neurons: an input layer, one or more hidden layers, and an output layer. Neurons in each layer are typically fully connected to the next layer, and hidden layers use nonlinear activation functions such as sigmoid, tanh, or ReLU.

Because of the multiple hidden layers and nonlinear activations, an MLP can represent and learn complex, non-linear relationships in data, making it suitable for tasks like image classification, regression, and general pattern recognition. MLPs are trained with gradient descent and backpropagation, which compute how much each weight contributed to the error and update all weights in all layers accordingly.

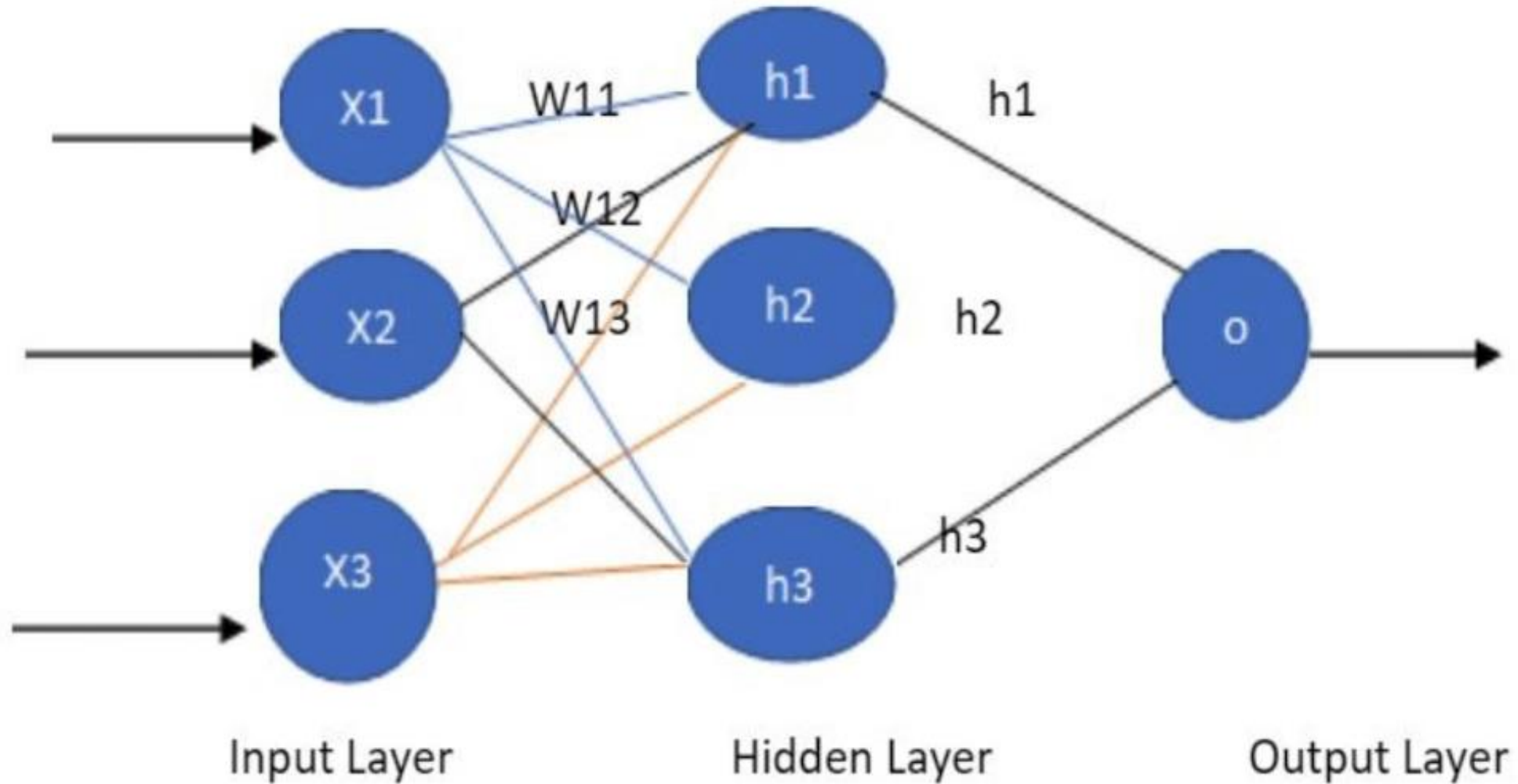


Diagram Of MultiLayer Perceptron Neural Network

3. PERCEPTRON & MULTI-LAYER PERCEPTRON (MLP)

All Rights Reserved. Author @ Rajendra Phani

- Perceptron (Single Layer)
- Linear classifier: $f(x) = \sigma(w^T x + b)$
- Only solves *linearly separable* problems (e.g., AND, OR — but not XOR).
- MLP (≥ 1 hidden layer)
- Universal function approximator (Cybenko, 1989).
- Example: 2-layer MLP:
 - $h = \sigma(W^{(1)}x + b^{(1)})$
 - $\hat{y} = \sigma(W^{(2)}h + b^{(2)})$
- Non-linearity via activation functions $\rightarrow$ solves XOR.

PERCEPTRON & MLP CODE

All Rights Reserved. Author @ Rajendra Phani

```
# Perceptron (linear binary classifier)
```

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(1, activation='sigmoid', input_shape=(2,))  
])
```

```
# MLP with 1 hidden layer (solves XOR)
```

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(4, activation='relu', input_shape=(2,)),  
    tf.keras.layers.Dense(1, activation='sigmoid')  
])
```

```
# Compile & train (XOR example)
```

```
X = tf.constant([[0,0], [0,1], [1,0], [1,1]], dtype=tf.float32)
```

```
y = tf.constant([[0], [1], [1], [0]], dtype=tf.float32)
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
model.fit(X, y, epochs=500, verbose=0)
```

```
print("XOR accuracy:", model.evaluate(X, y, verbose=0)[1])
```

All Rights Reserved. Author @ Rajendra Phani

ACTIVATION FUNCTIONS

All Rights Reserved. Author @ Rajendra Phani

Activation functions in deep learning are mathematical functions applied to a neuron's weighted input sum to produce its output, introducing non-linearity that allows neural networks to model complex patterns beyond simple linear transformations. They determine whether a neuron activates and enable backpropagation by providing gradients for weight updates during training. Without them, even deep networks would function like basic linear models, limiting their ability to learn from real-world data.

ReLU

ReLU, or Rectified Linear Unit, outputs the input directly if positive ($\max(0, x)$) and zero otherwise, promoting sparsity by deactivating negative inputs. It accelerates training, avoids vanishing gradients, and is computationally efficient for hidden layers in most networks.

Sigmoid

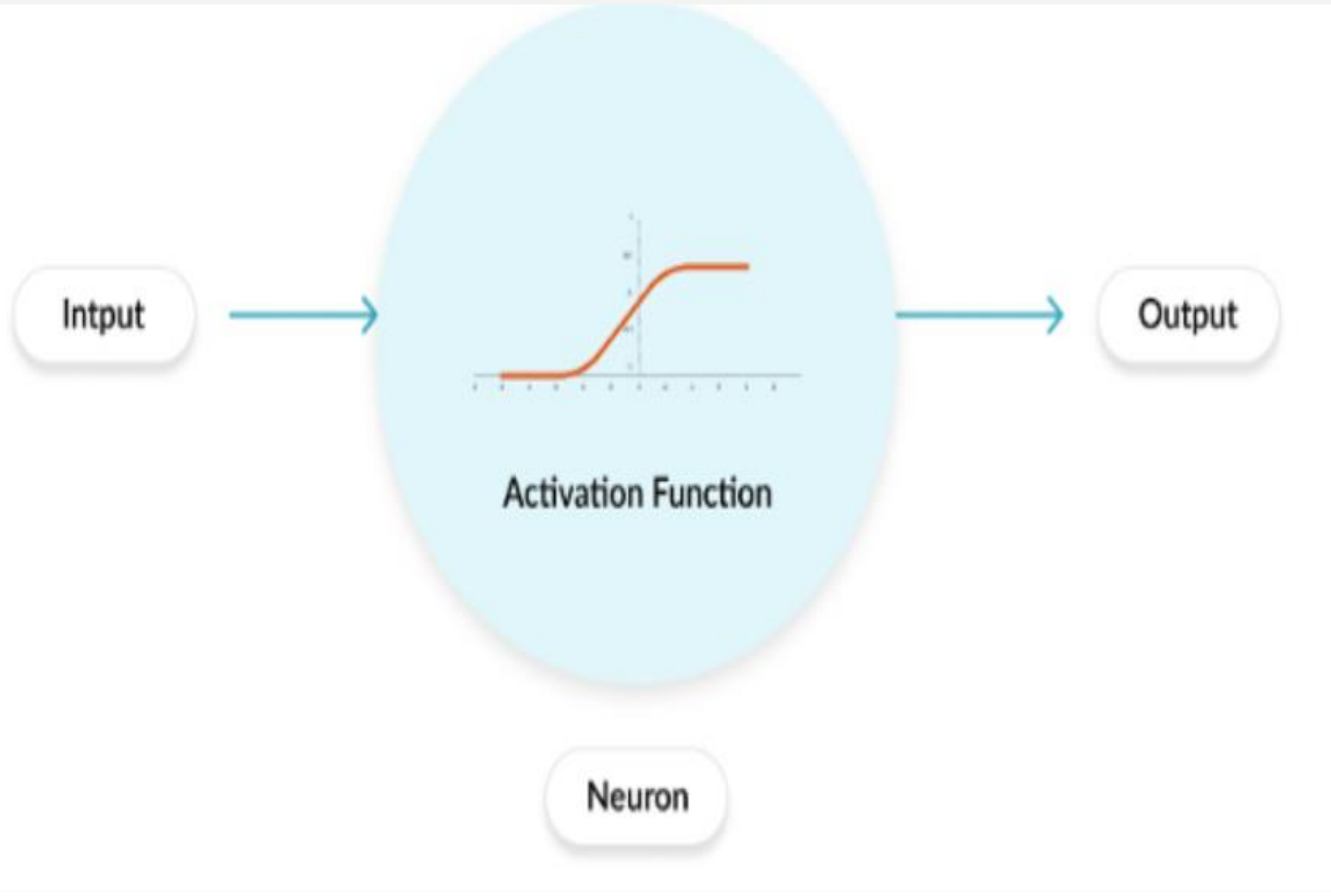
Sigmoid maps inputs to a range between 0 and 1 via $\sigma(x) = \frac{1}{1 + e^{-x}}$, making it ideal for binary classification outputs like probabilities. However, it suffers from vanishing gradients for large or small inputs, slowing deep network training.

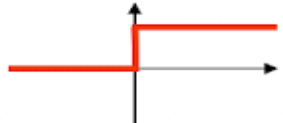
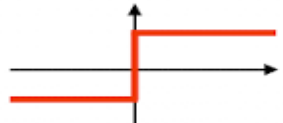
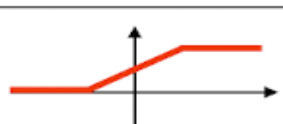
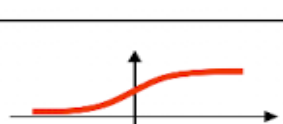
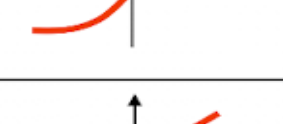
Tanh

Tanh, or hyperbolic tangent, outputs values between -1 and 1 ($\tanh(x)$), centering data around zero for better gradient flow than sigmoid. It suits hidden layers but can still face vanishing gradient issues in deep architectures.

Softmax

Softmax converts a vector of raw scores into probabilities summing to 1, using $\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ for multi-class classification in output layers. It highlights the largest input as the predicted class while normalizing others.



Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \max(0, z) + \ln(1 + e^z)$	Multi-layer Neural Networks	

4. ACTIVATION FUNCTIONS

All Rights Reserved. Author @ Rajendra Phani

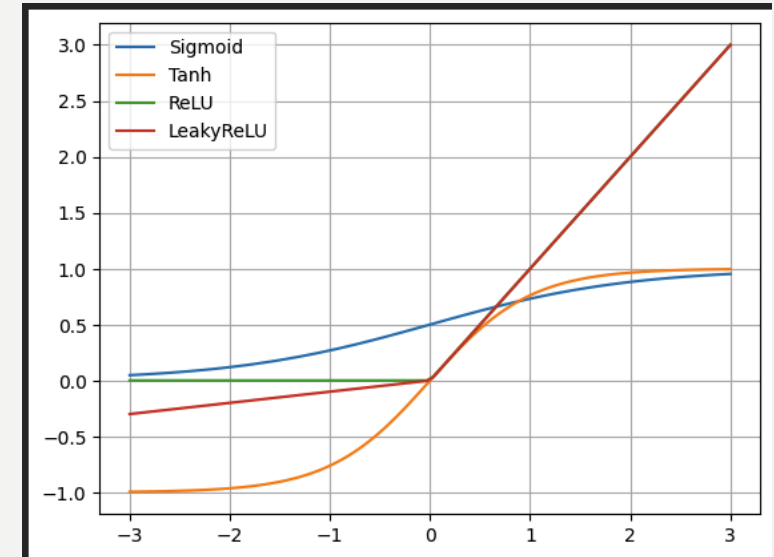
Function	Formula	Pros	Cons
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Smooth, (0, 1) output	Vanishing gradients, not zero-centered
Tanh		Zero-centered, stronger gradients	Vanishing gradients
ReLU		Fast, no vanishing grad (for $z > 0$)	Dead neurons ($z < 0$)
Leaky ReLU	$\text{LReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$	Fixes dead ReLU	α hyperparam
Softmax (output layer)	$\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	Multiclass prob. dist.	—

ACTIVATION FUNCTIONS CODE

All Rights Reserved. Author @ Rajendra Phani

```
import matplotlib.pyplot as plt
x = tf.linspace(-3.0, 3.0, 100)

plt.plot(x, tf.nn.sigmoid(x), label='Sigmoid')
plt.plot(x, tf.nn.tanh(x), label='Tanh')
plt.plot(x, tf.nn.relu(x), label='ReLU')
plt.plot(x, tf.nn.leaky_relu(x, alpha=0.1), label='LeakyReLU')
plt.legend(); plt.grid()
plt.show()
```



FORWARD PROPAGATION

All Rights Reserved. Author @ Rajendra Phani

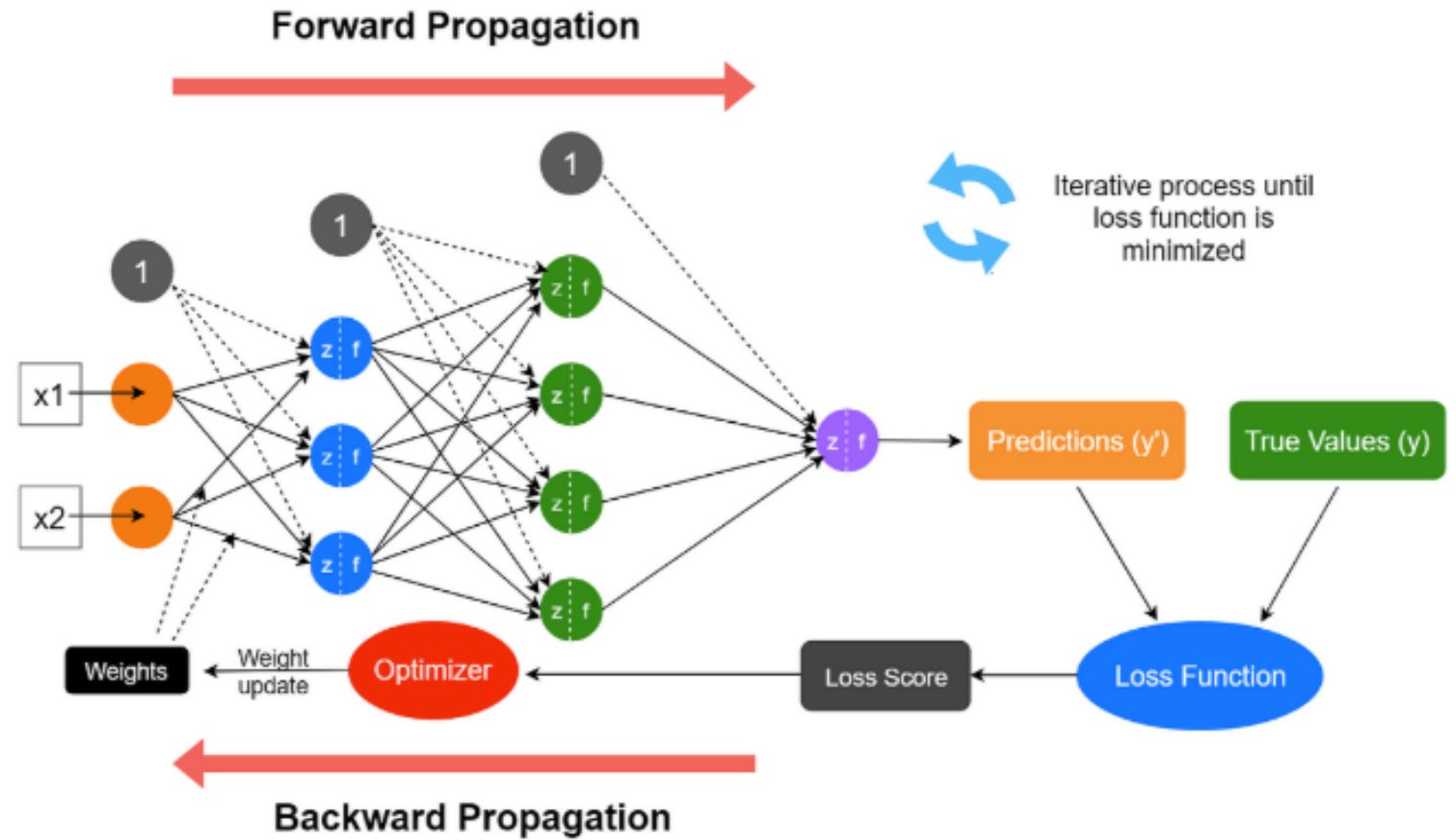
Forward propagation is the process in neural networks where input data flows sequentially through each layer, undergoing linear transformations (weighted sums plus biases) and nonlinear activation functions to produce a final output prediction.

Process Overview

Data starts at the input layer and moves forward layer by layer. Each neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function like ReLU or sigmoid before passing the result to the next layer. This stores intermediate activations needed for subsequent training steps.

Key Importance

Forward propagation generates predictions essential for comparing against true labels to compute loss during training. It forms the foundation for backpropagation, enabling weight updates, and supports efficient model evaluation on new data. Additionally, its computational efficiency suits real-time applications like recommendations or autonomous systems.



5. FORWARD PROPAGATION

All Rights Reserved. Author @ Rajendra Phani

- Computing output layer-by-layer:
- For layer ℓ :
- $z^{[\ell]} = W^{[\ell]}a^{[\ell-1]} + b^{[\ell]} = g^{[\ell]}(z^{[\ell]})$
- $a^{[0]} = x$ (input)
- $\hat{y} = a^{[L]}$ (final output, L = total layers)

FORWARD PROPAGATION CODE

All Rights Reserved. Author @ Rajendra Phani

```
# Manual forward pass (for teaching)
x = tf.random.normal((1, 3)) # batch=1, input=3
W1 = tf.Variable(tf.random.normal((3, 4)))
b1 = tf.Variable(tf.zeros(4))
W2 = tf.Variable(tf.random.normal((4, 2)))
b2 = tf.Variable(tf.zeros(2))

h = tf.nn.relu(tf.matmul(x, W1) + b1) # Layer 1
y_hat = tf.nn.softmax(tf.matmul(h, W2) + b2) # Output layer

print("Input:", x.numpy().flatten())
print("Output (prob):", y_hat.numpy().flatten())
```

LOSS FUNCTIONS

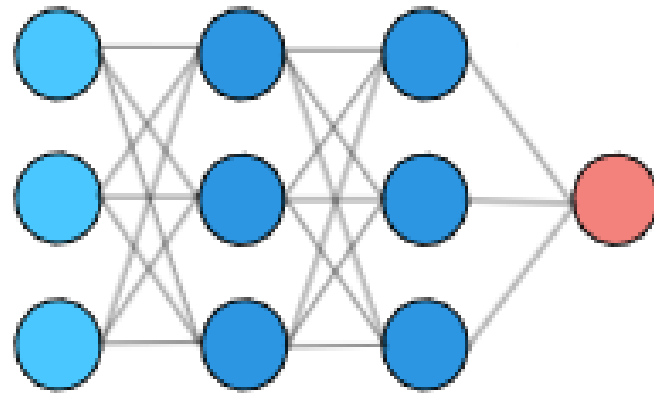
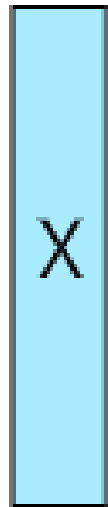
All Rights Reserved. Author @ Rajendra Phani

A loss function in deep learning is a mathematical function that quantifies the difference or error between the model's predicted output and the actual target value. Its purpose is to measure how well or poorly the model is performing by giving a numerical value representing prediction error.

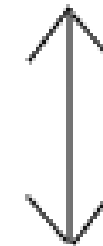
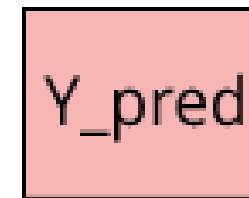
During training, the loss function guides the optimization process by providing a signal to adjust the model's parameters (weights and biases) to minimize this error, thereby improving the model's predictions. It essentially directs the learning algorithm on how to update parameters to reduce discrepancies between predicted and true values.

Additionally, loss functions help in evaluating model performance, balancing bias and variance, and influencing the way models learn from data depending on the task and nature of errors. Different loss functions are suited for different types of problems such as regression or classification, and their choice significantly impacts model convergence and accuracy.

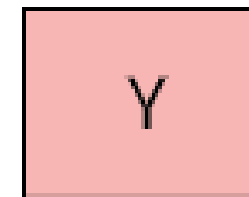
Input Data



Predicted Output



$$\text{Loss} = J(Y_{\text{pred}}, Y)$$



True Output

6. LOSS FUNCTIONS

All Rights Reserved. Author @ Rajendra Phani

Task	Loss Function	Formula
Regression	Mean Squared Error (MSE)	$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$
Binary Class.	Binary Cross-Entropy	$\mathcal{L} = -\frac{1}{m} \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
Multiclass	Categorical Cross-Entropy	$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$

LOSS FUNCTIONS

All Rights Reserved. Author @ Rajendra Phani

```
y_true = tf.constant([[1, 0], [0, 1]], dtype=tf.float32) # 2-class labels  
y_pred = tf.constant([[0.9, 0.1], [0.2, 0.8]], dtype=tf.float32)
```

```
# Categorical crossentropy (multi-class)  
cce = tf.keras.losses.CategoricalCrossentropy()  
loss = cce(y_true, y_pred)  
print("CCE Loss:", loss.numpy())
```

```
# MSE (regression)  
mse = tf.keras.losses.MeanSquaredError()  
loss = mse([3.0], [2.5])  
print("MSE:", loss.numpy())
```


GRADIENT DESCENT & BACKPROPAGATION

All Rights Reserved. Author @ Rajendra Phani

Gradient descent is an optimization algorithm that minimizes a neural network's loss function by iteratively adjusting model parameters like weights and biases in the direction of the steepest descent, based on the negative gradient. Backpropagation (short for backward propagation of errors) computes these gradients efficiently by propagating the error from the output layer back through the network layer by layer, using the chain rule of calculus.

How They Work Together

Training begins with forward propagation, where input data passes through the network to produce a prediction and calculate the loss. Backpropagation then performs the backward pass to determine how much each weight contributed to the error, yielding gradients for every parameter. Gradient descent applies these gradients to update parameters via the rule $\theta = \theta - \eta \cdot \nabla_{\theta} \text{Loss}$, where η is the learning rate and $\nabla_{\theta} \text{Loss}$ is the gradient, repeating over epochs until convergence.

Key Process Steps

- Forward pass: Compute predictions and loss.
- Backward pass (backpropagation): Calculate gradients using chain rule, starting from output.
- Parameter update (gradient descent): Adjust weights/biases to reduce loss.

This synergy enables deep networks to learn complex patterns in supervised tasks like classification

Cost

All Rights Reserved. Author @ Rajendra Phani

Initial
Weight

Gradient

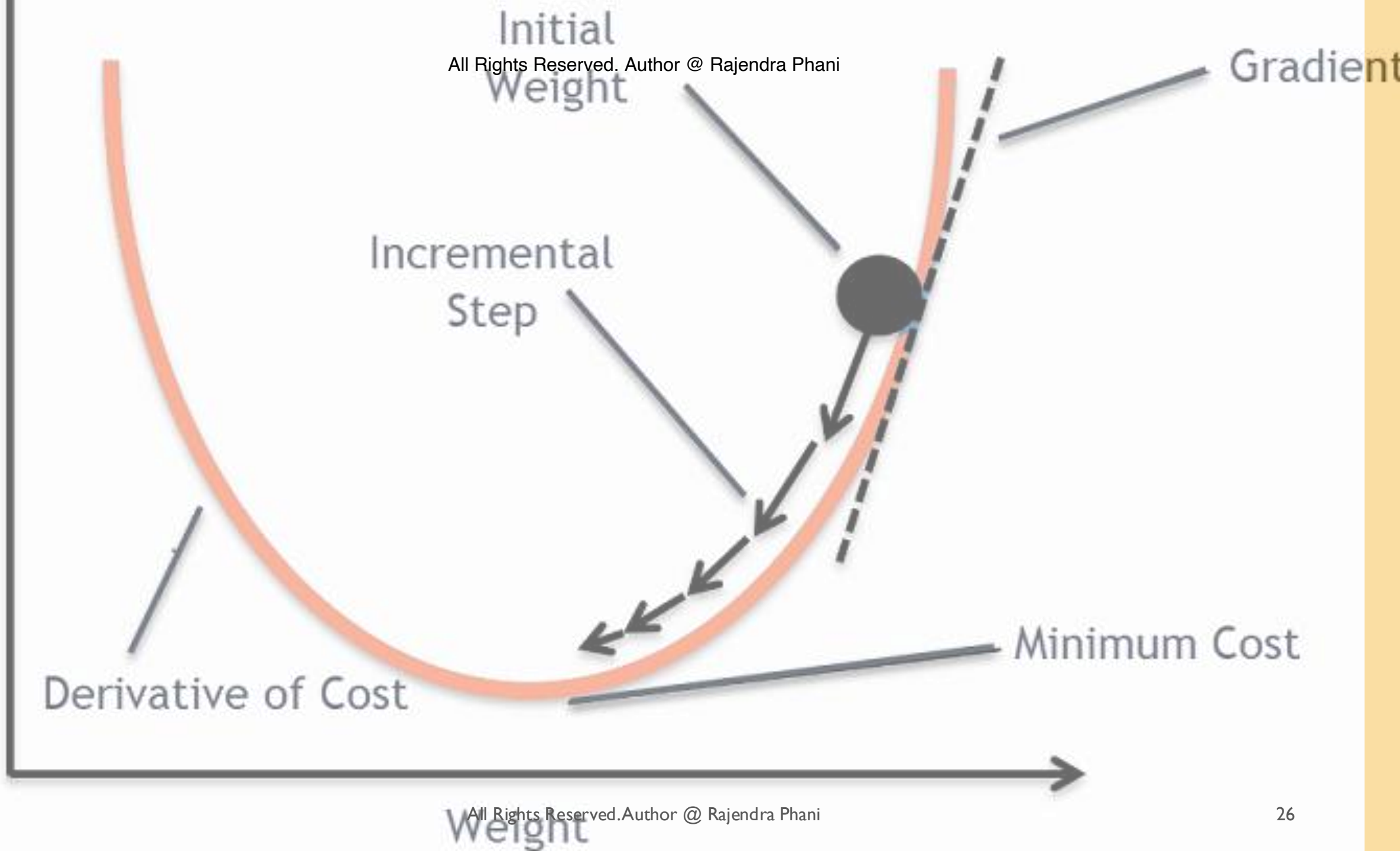
Incremental
Step

Derivative of Cost

Minimum Cost

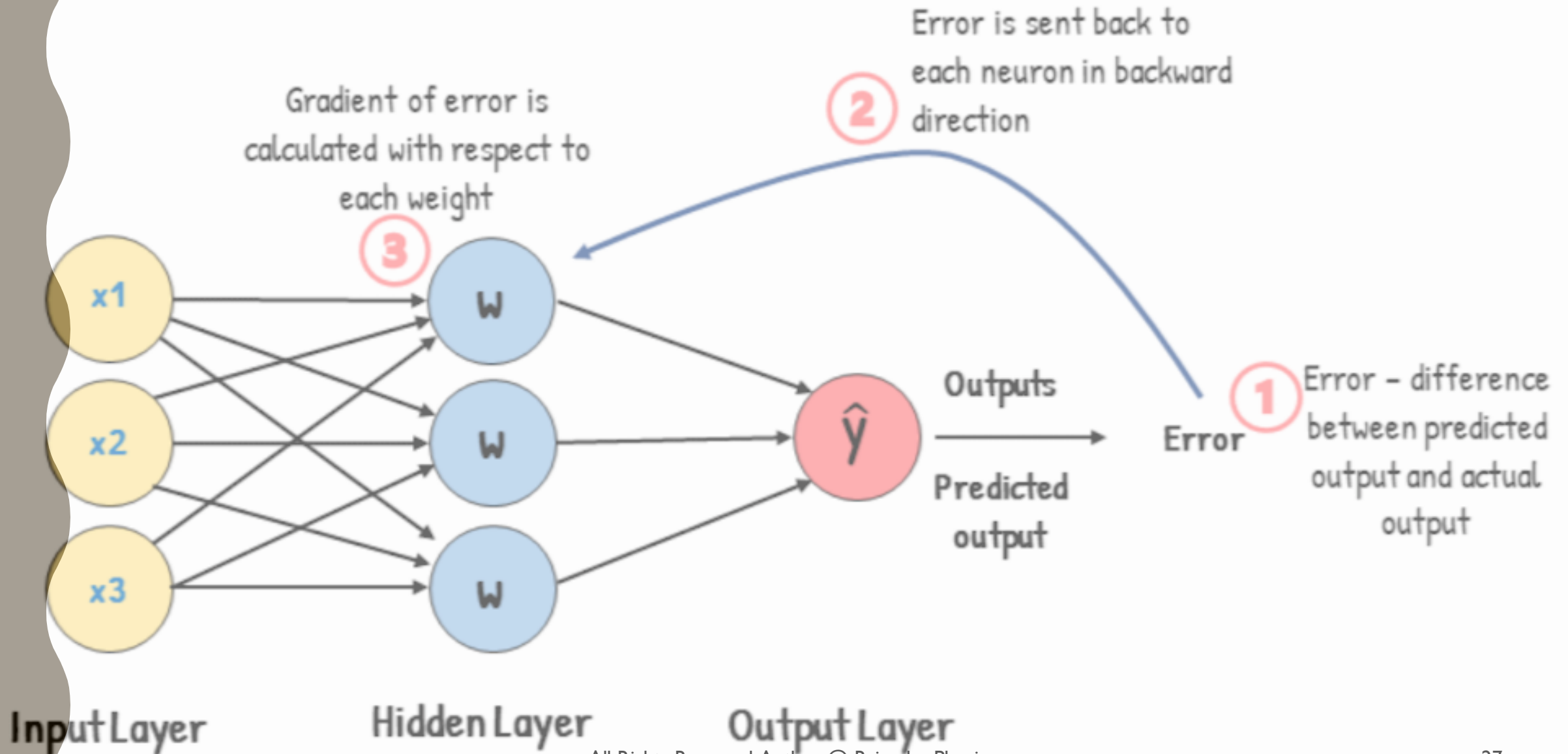
All Rights Reserved. Author @ Rajendra Phani

Weight



Backpropagation

All Rights Reserved. Author @ Rajendra Phani



All Rights Reserved. Author @ Rajendra Phani

7. GRADIENT DESCENT & BACKPROPAGATION

All Rights Reserved. Author @ Rajendra Phani

- Gradient Descent Update
- $\theta := \theta - \alpha \nabla_{\theta} \mathcal{L}$
- α : learning rate
- $\nabla_{\theta} \mathcal{L}$: gradient of loss w.r.t. parameters
- Backpropagation (chain rule applied efficiently)
- Compute gradients from output $\rightarrow$ input:
- $\frac{\partial \mathcal{L}}{\partial W^{[\ell]}} = \frac{\partial \mathcal{L}}{\partial z^{[\ell]}} \cdot \frac{\partial z^{[\ell]}}{\partial W^{[\ell]}} = \delta^{[\ell]} (a^{[\ell-1]})^T$
- where $\delta^{[\ell]} = \frac{\partial \mathcal{L}}{\partial z^{[\ell]}}$
- Key Insight: Reuses intermediate computations $\rightarrow O(n)$ instead of $O(n^2)$.

GRADIENT DESCENT & BACKPROPAGATION CODE

All Rights Reserved. Author @ Rajendra Phani

```
x = tf.constant([[1.0, 2.0]])
y_true = tf.constant([[1.0]])
W = tf.Variable([[0.5], [0.5]])
b = tf.Variable(0.0)

with tf.GradientTape() as tape:
    z = tf.matmul(x, W) + b
    y_pred = tf.nn.sigmoid(z)
    loss = tf.keras.losses.binary_crossentropy(y_true, y_pred)

# Compute gradients
grads = tape.gradient(loss, [W, b])
print("∂L/∂W =", grads[0].numpy(), ", ∂L/∂b =", grads[1].numpy())

# Manual update (SGD)
lr = 0.1
W.assign_sub(lr * grads[0])
b.assign_sub(lr * grads[1])
```

```
∂L/∂W = [[-0.1824255]
 [-0.364851  ]], ∂L/∂b = -0.1824255
<tf.Variable 'UnreadVariable' shape=() dtype=float32, numpy=0.018242551013827324>
```

Optimization algorithms in deep learning are methods used to adjust model parameters iteratively to minimize the loss function, enabling effective training of neural networks.

Types in Deep Learning

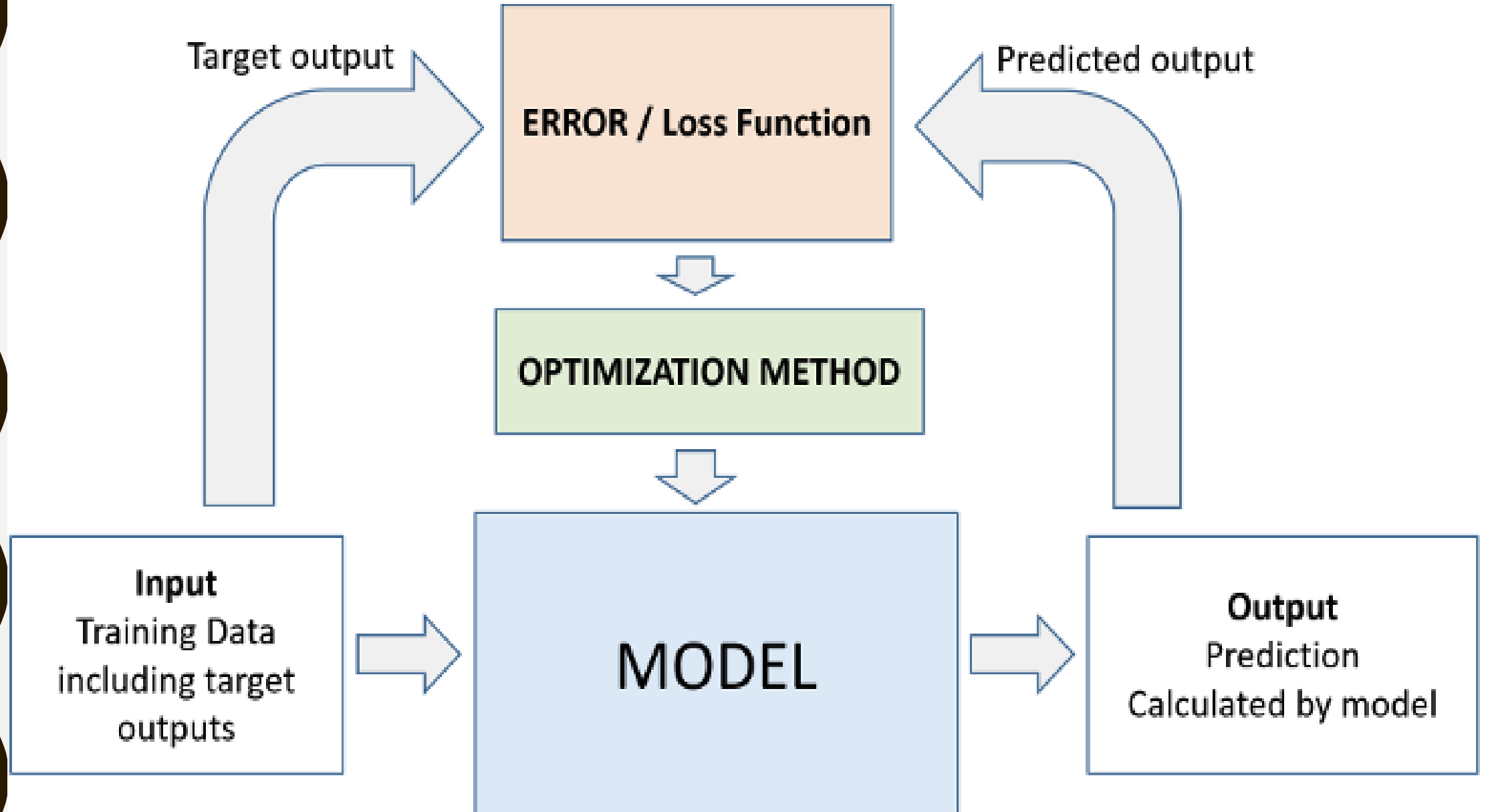
Common types include Gradient Descent variants like batch, Stochastic Gradient Descent (SGD), and Mini-batch GD, which process data differently for efficiency on large datasets. Adaptive methods such as AdaGrad, RMSprop, AdaDelta, and Adam adjust learning rates per parameter based on gradient history to handle sparse data and speed convergence. Other advanced ones like Nesterov Accelerated Gradient (NAG) and RAdam build on these for better stability and generalization.

SGD Overview

Stochastic Gradient Descent (SGD) updates parameters using gradients from single examples or small batches, introducing randomness to escape local minima faster than full-batch GD, though it can be noisy. It suits large datasets by accelerating training but requires careful learning rate tuning. Momentum-enhanced SGD smooths updates for quicker convergence.

Adam Overview

Adam (Adaptive Moment Estimation) combines momentum and RMSprop ideas, using first and second moments of gradients to adapt learning rates per parameter dynamically, with bias correction for stability. It converges quickly, needs less tuning, and works robustly across architectures, making it a default choice despite occasional generalization issues.



8. OPTIMIZATION ALGORITHMS

All Rights Reserved. Author @ Rajendra Phani

Algorithm	Update Rule	Key Idea
SGD	$\theta := \theta - \alpha \nabla \mathcal{L}(\theta; x^{(i)}; y^{(i)})$	No momentum; noisy
SGD + Momentum	$v := \beta v + (1 - \beta) \nabla \mathcal{L}$ $\theta := \theta - \alpha v$	Smooths path; avoids oscillations
Adam	Adaptive 1st & 2nd moment: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ $\hat{m}_t$	Combines momentum + RMSProp; adaptive LR per param

Default: Adam $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$

OPTIMIZATION ALGORITHMS CODE

All Rights Reserved. Author @ Rajendra Phani

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1, input_shape=(2,))])
```

```
# Compare optimizers (set in compile)
```

```
model.compile(
```

```
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), # default
```

```
    loss='mse'
```

```
)
```

```
# Others:
```

```
# optimizer='sgd' → basic SGD
```

```
# optimizer=tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
```

```
# optimizer=tf.keras.optimizers.RMSprop()
```

OVERFITTING & REGULARIZATION

All Rights Reserved. Author @ Rajendra Phani

Overfitting in deep learning occurs when a model learns the training data too closely, including noise and irrelevant details, leading to excellent performance on training data but poor generalization to unseen data.

Overfitting Explained

Models overfit due to excessive complexity, small training datasets, noisy data, or prolonged training, causing high variance and memorization rather than pattern recognition. This results in smooth decision boundaries on training sets but erratic performance on test sets.

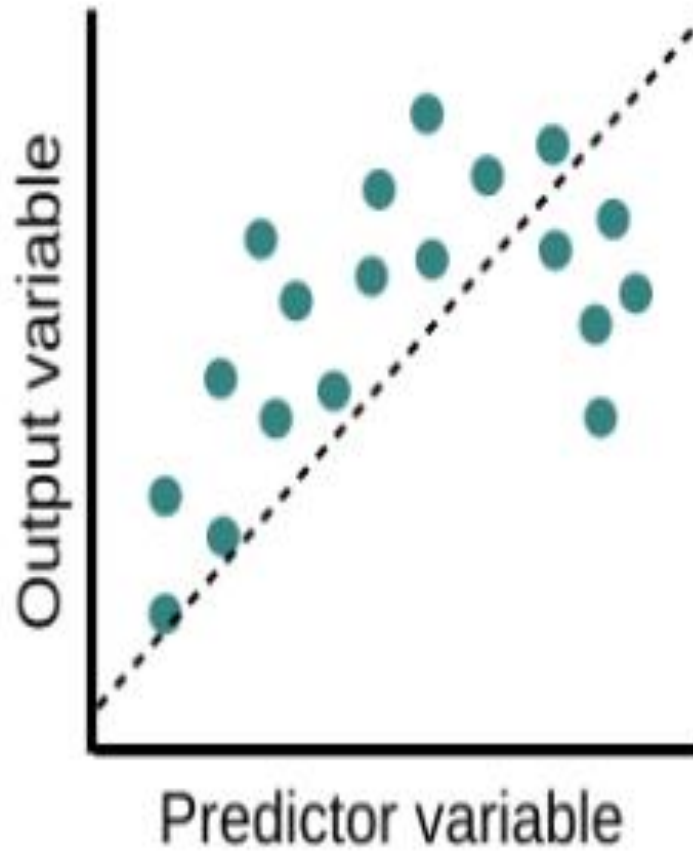
Regularization Overview

Regularization prevents overfitting by adding penalties to the loss function, constraining model complexity and encouraging simpler, more generalizable parameters. It trades some bias for reduced variance, balancing fit on training data with performance on new data.

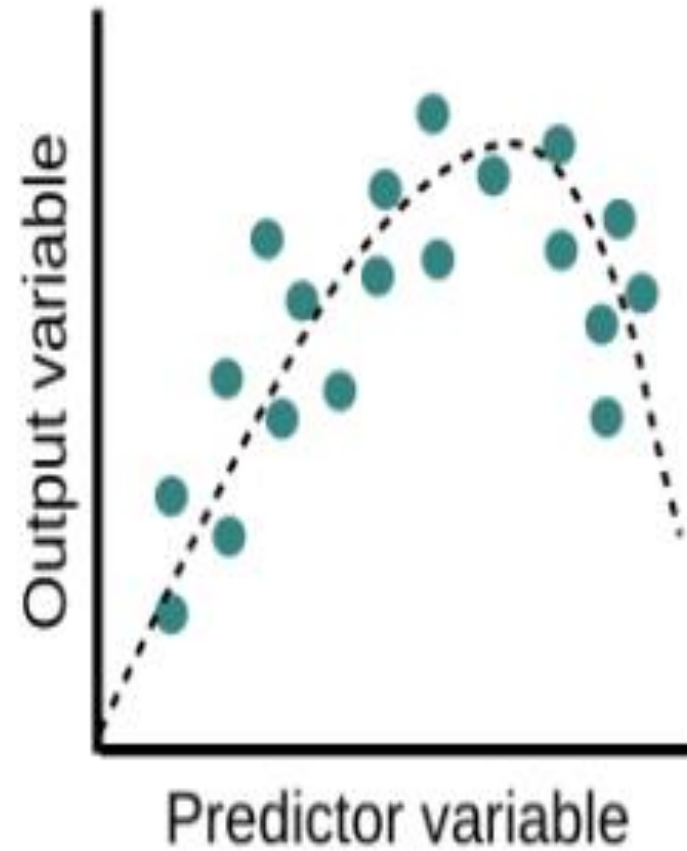
Key Techniques

- **L1 (Lasso):** Penalizes the absolute value of weights, promoting sparsity by driving less important ones to zero, which simplifies the model.
- **L2 (Ridge):** Penalizes the squared magnitude of weights, shrinking them evenly to reduce sensitivity to outliers.
- **Dropout:** Randomly deactivates neurons during training, creating varied sub-networks to prevent co-dependency and mimic ensembles.

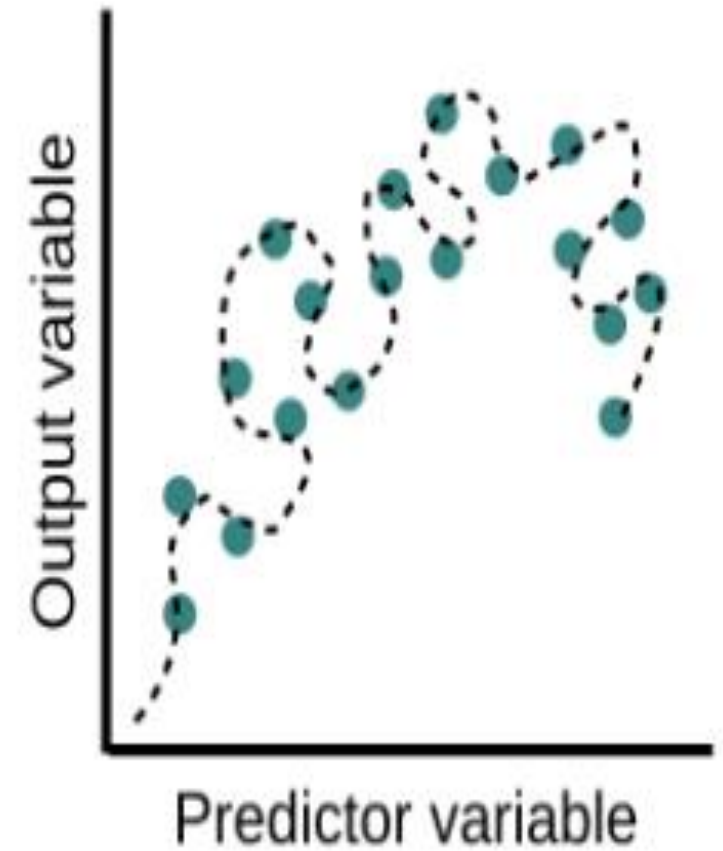
Underfit



Optimal



Overfit



9. OVERFITTING & REGULARIZATION

All Rights Reserved. Author @ Rajendra Phani

- Symptoms: Low train error, high val/test error.

Techniques:

- L2 Regularization (Weight Decay):
- $\mathcal{L}_{total} = \mathcal{L} + \lambda \sum_{\ell} \|W^{[\ell]}\|_F^2$
- L1 Regularization: Encourages sparsity: $\lambda \sum |W_{ij}|$
- Dropout: Randomly set fraction p of activations to 0 during training.
 - At test time: scale by p or use full net (no dropout).
- Early Stopping: Monitor val loss; stop when it starts increasing.
- Use Dropout + L2 together for best results.

REGULARIZATION (DROPOUT, L2)

All Rights Reserved. Author @ Rajendra Phani

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu',
                           kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy')
```

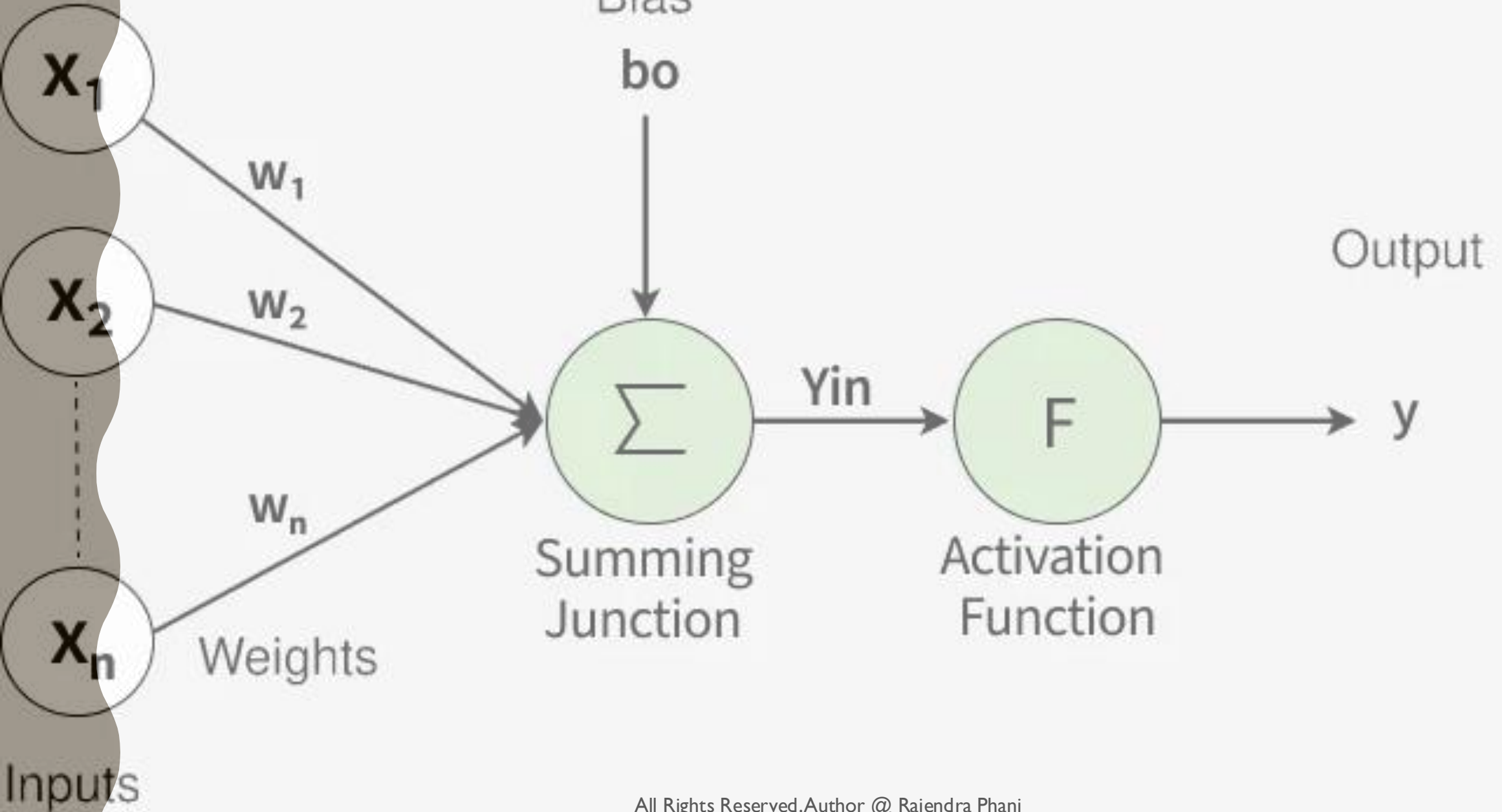
Weight initialization in deep learning refers to setting the initial values of a neural network's weights (and often biases) before training begins, providing a starting point for optimization algorithms like gradient descent.

Importance

Proper initialization prevents issues such as vanishing or exploding gradients, where signals become too small or too large across layers, slowing convergence or causing instability. It ensures balanced activation scales and gradient flow, improving training speed and final model quality.

Common Techniques

- **Zero Initialization:** Sets all weights to zero, but this leads to symmetric neurons learning identical features, making it ineffective for weights (though usable for biases).
- **Random Initialization:** Uses small random values, often from a normal or uniform distribution, to break symmetry.
- **Xavier/Glorot:** Scales weights by the square root of fan-in (input size) or average of fan-in and fan-out, ideal for sigmoid or tanh activations.
- **He Initialization:** Doubles the Xavier variance for ReLU activations, maintaining signal variance through layers.



10. WEIGHT INITIALIZATION

All Rights Reserved. Author @ Rajendra Phani

Method	Formula	Use Case
Xavier/Glorot	$W \sim u\left(-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}\right)$	Tanh/sigmoid
He Initialization	$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$	ReLU and variants

WEIGHT INITIALIZATION

All Rights Reserved. Author @ Rajendra Phani

He initialization (default for ReLU in Dense/Conv2D)

```
layer = tf.keras.layers.Dense(10,  
                                activation='relu',  
                                kernel_initializer='he_normal')
```

Xavier (for tanh)

```
layer = tf.keras.layers.Dense(10,  
                                activation='tanh',  
                                kernel_initializer='glorot_uniform')
```

WEIGHT INITIALIZATION CODE

All Rights Reserved. Author @ Rajendra Phani

```
# He initialization (default for ReLU in Dense/Conv2D)
```

```
layer = tf.keras.layers.Dense(10,  
                                activation='relu',  
                                kernel_initializer='he_normal')
```

```
# Xavier (for tanh)
```

```
layer = tf.keras.layers.Dense(10,  
                                activation='tanh',  
                                kernel_initializer='glorot_uniform')
```

BATCH NORMALIZATION

All Rights Reserved. Author @ Rajendra Phani

Batch normalization is a technique in deep learning that normalizes the inputs to each layer within a mini-batch during training, stabilizing the learning process by addressing internal covariate shift—the change in distribution of layer inputs as preceding layers update.

How It Works

For a mini-batch of activations , compute the batch mean and variance . Normalize each activation as , where ensures numerical stability. Then scale and shift using learnable parameters and : , allowing the network to adjust the distribution if needed.

$$\begin{aligned}\hat{x}_i &= \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \\ y_i &= \gamma \hat{x}_i + \beta, \end{aligned}$$

Key Benefits

- Accelerates training by enabling higher learning rates and reducing sensitivity to weight initialization.
- Mitigates vanishing/exploding gradients and adds mild regularization, often reducing the need for dropout.
- Improves stability for deeper networks by keeping activations in a consistent range.

11. BATCH NORMALIZATION

All Rights Reserved. Author @ Rajendra Phani

- Normalizes layer inputs *per mini-batch*:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} y^{(k)} = \gamma \hat{x}^{(k)} + \beta$$

- μ_B, σ_B^2 : mean & var of batch
- γ, β : learnable scale & shift

Benefits:

- Faster training
- Less sensitive to initialization
- Acts as regularizer

Use *after* activation (or before — debated), but before dropout.

BATCH NORMALIZATION CODE

All Rights Reserved. Author @ Rajendra Phani

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(64, activation='relu'),  
    tf.keras.layers.BatchNormalization(), # ← after activation  
    tf.keras.layers.Dropout(0.3),  
    tf.keras.layers.Dense(1, activation='sigmoid')  
])
```

CONVOLUTIONAL NEURAL NETWORKS (CNNs)

All Rights Reserved. Author @ Rajendra Phani

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed primarily for processing grid-like data such as images, where it automatically learns hierarchical features through convolution operations.

Key Components

CNNs consist of convolutional layers that apply learnable filters (kernels) to input data, sliding across the image to produce feature maps detecting edges, textures, or patterns. Pooling layers follow to downsample these maps, reducing dimensions while retaining essential information, often using max pooling to select the strongest activations. Activation functions like ReLU introduce non-linearity, and fully connected layers at the end combine features for classification or prediction.

How CNNs Work

Input images pass through convolutional layers, where filters compute dot products with local regions to generate activation maps, enabling parameter sharing for efficiency and translation invariance. Subsequent pooling and additional convolutions build increasingly abstract representations, from simple edges to complex objects. The process repeats across multiple layers, culminating in output for tasks like classification.

Applications

CNNs excel in image classification, object detection, segmentation, and video analysis, powering computer vision in autonomous driving, medical imaging, and recommender systems. They also extend to audio, time-series, and natural language processing when data is structured grid-like.

A Convolutional Neural Network (CNN) is a deep learning model designed for processing grid-like data such as images, featuring a stacked architecture of specialized layers that progressively extract features, reduce dimensionality, and produce classifications or predictions.

Core Layers

Convolutional Layer

This foundational layer applies learnable filters (kernels) that slide over the input image, performing dot products to detect local patterns like edges, textures, and shapes, generating feature maps. Multiple filters create depth in these maps, capturing diverse features at different spatial positions.

Activation Layer

Typically using ReLU, this layer introduces nonlinearity after convolution, zeroing out negative values to enable learning of complex patterns while accelerating training. It ensures the model avoids linear transformations, vital for handling real-world image complexities.

Pooling Layer

This downsampling layer reduces feature map dimensions by operations like max-pooling, retaining dominant features while discarding less critical details, which cuts computation and adds translation invariance. Common in sequences after convolutions, it helps control overfitting.

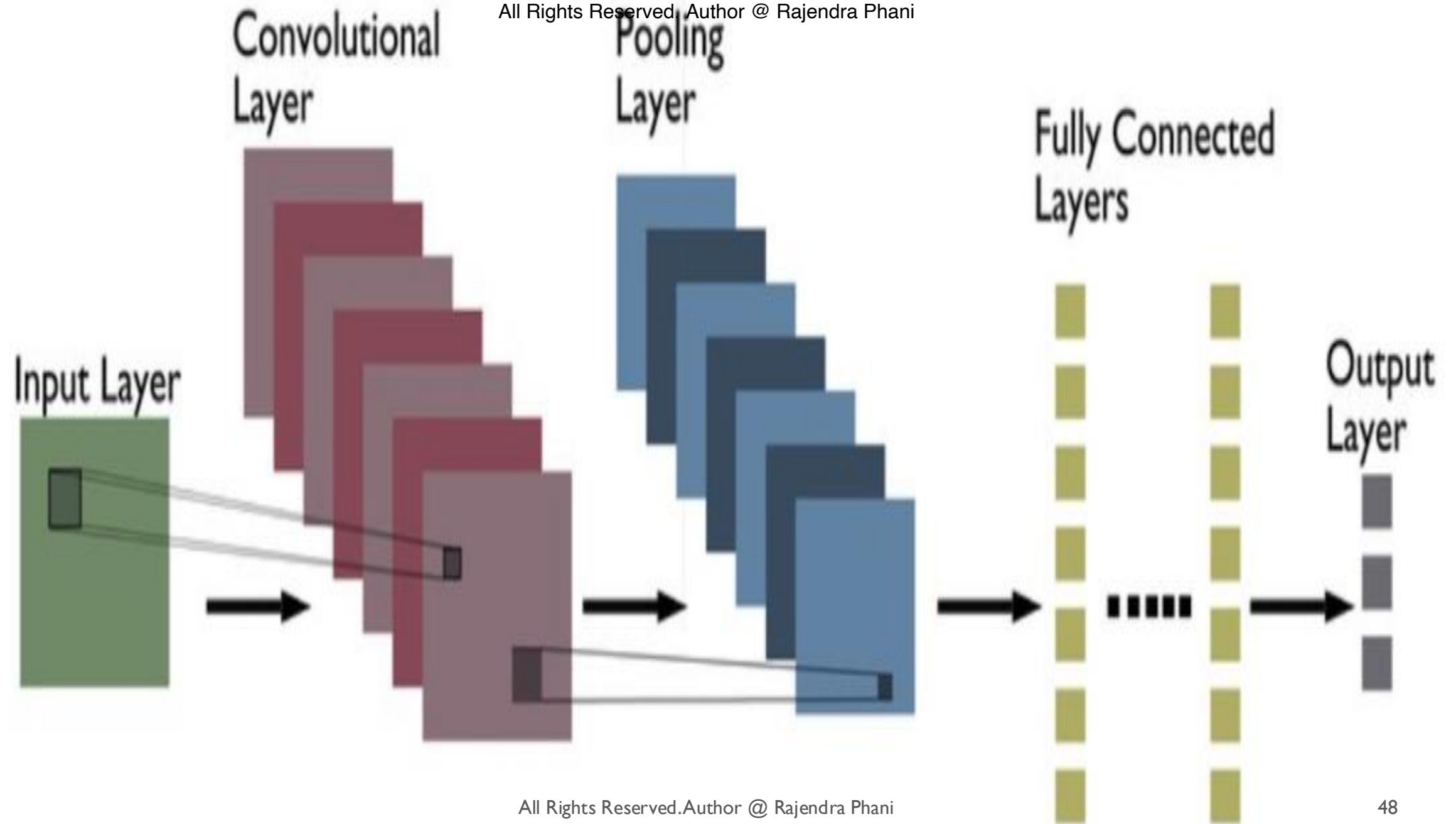
Classification Stages

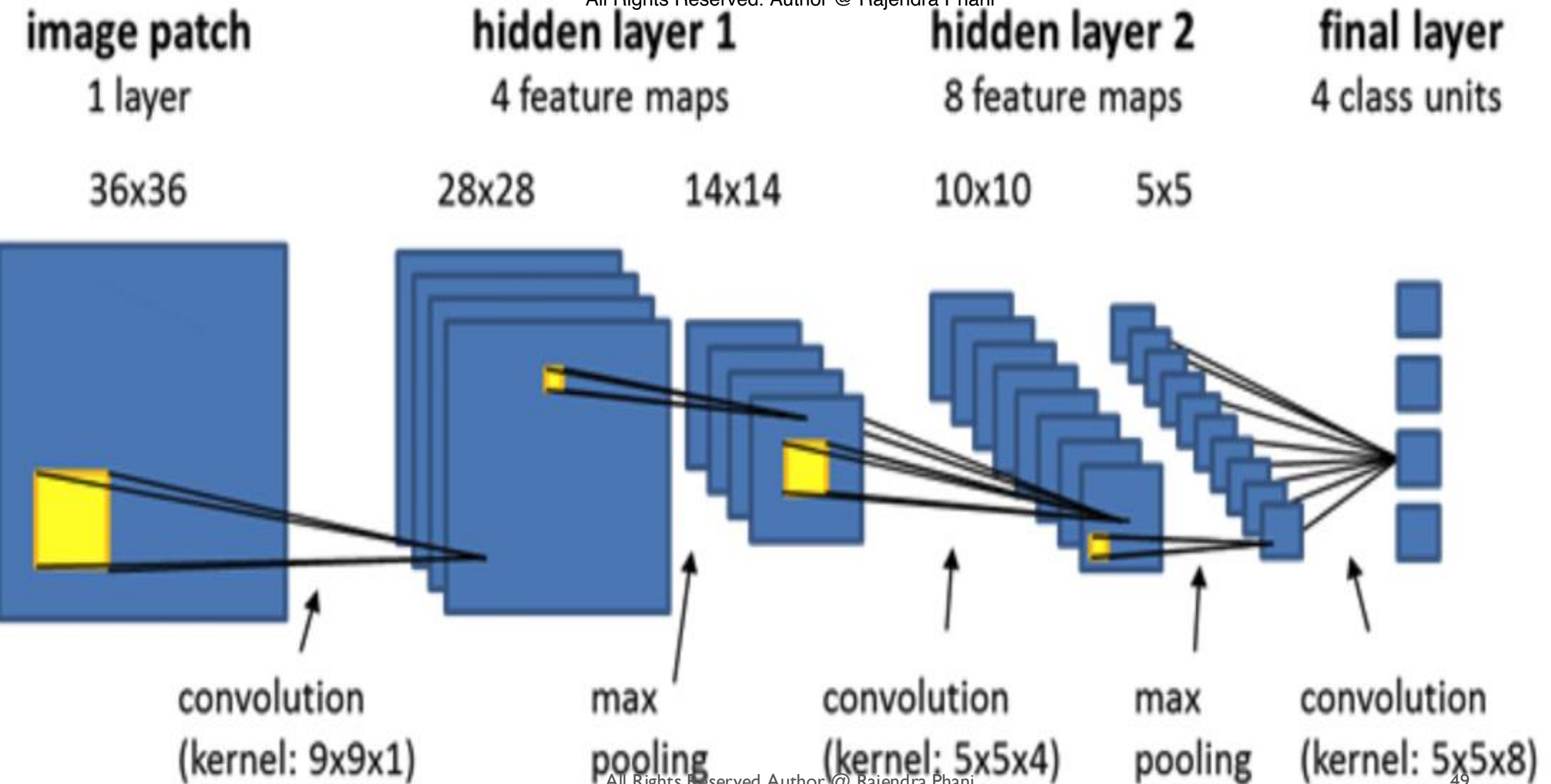
Flattening and Fully Connected Layers

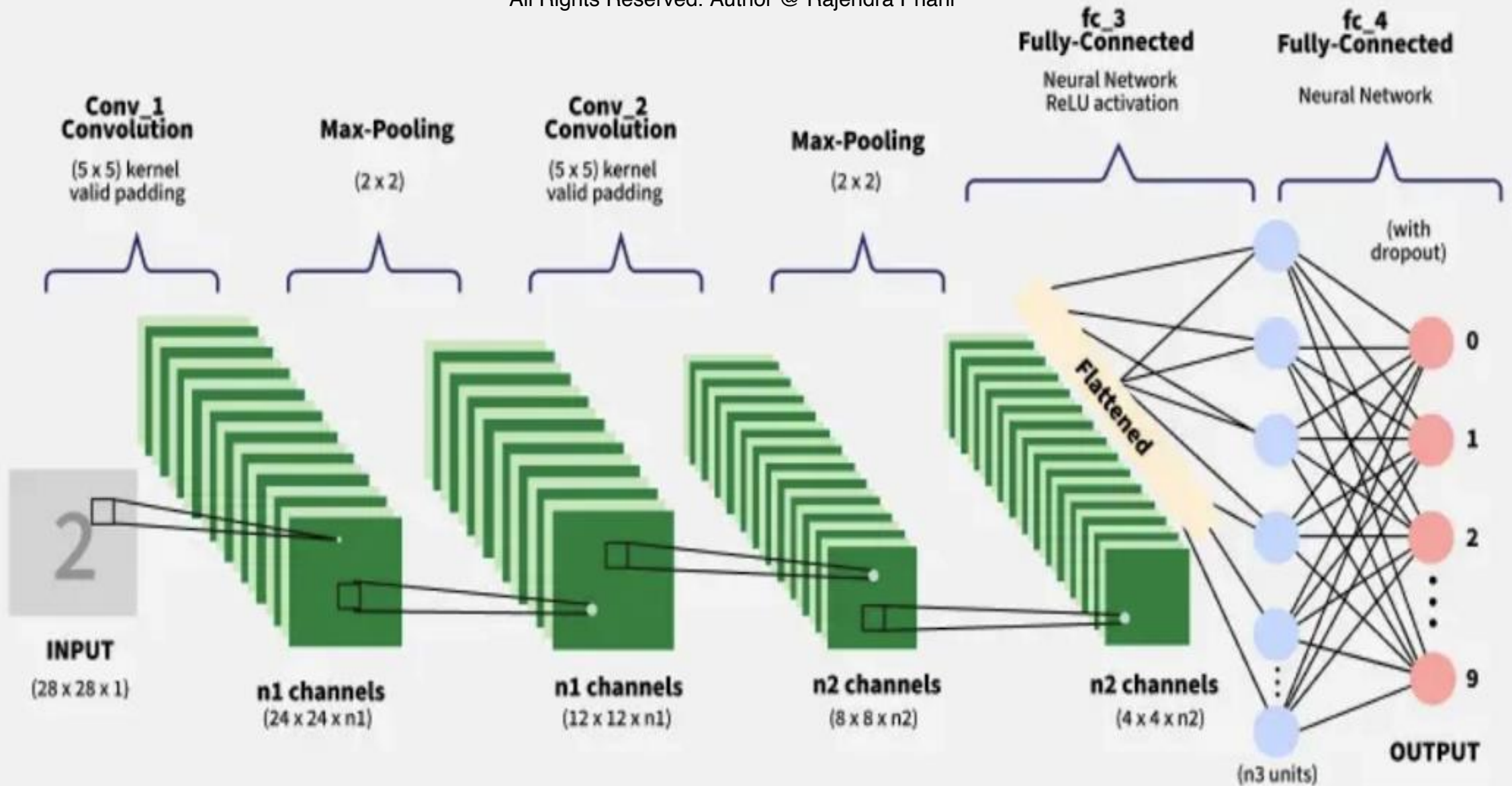
Feature maps flatten into a vector before entering dense (fully connected) layers, where neurons integrate high-level features for pattern relationships and decision-making. These layers, often multiple, transition from spatial to abstract representations.

Output Layer

Using softmax for multi-class or sigmoid for binary tasks, this final layer converts scores into probabilities, yielding the prediction (e.g., class label). It completes the forward pass, enabling backpropagation during training.

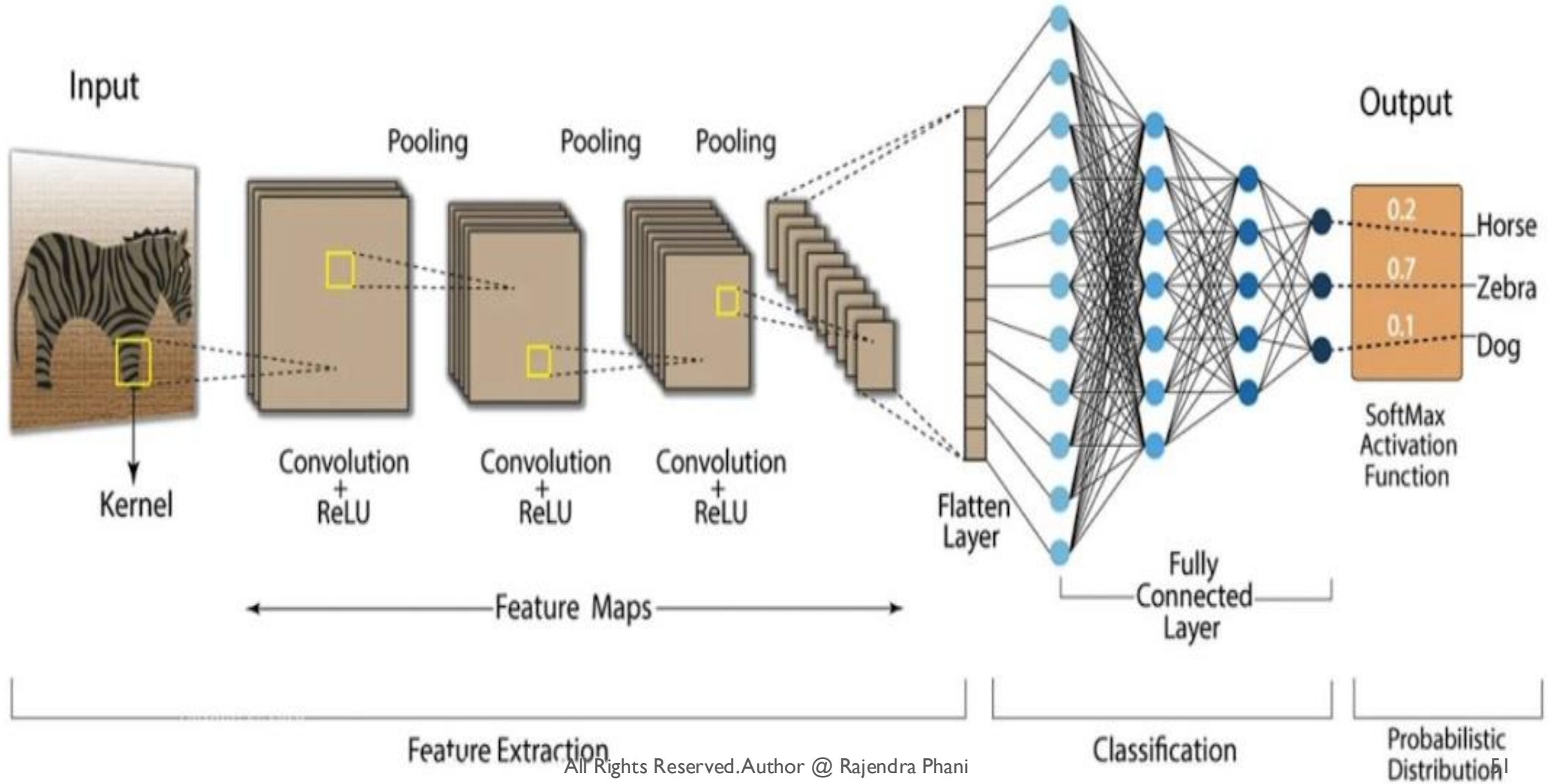






Convolution Neural Network (CNN)

All Rights Reserved. Author @ Rajendra Phani



All Rights Reserved. Author @ Rajendra Phani

12. CONVOLUTIONAL NEURAL NETWORKS (CNNs)

All Rights Reserved. Author @ Rajendra Phani

- Why CNNs?
- Exploit spatial locality & translation invariance.
- Reduce parameters vs. dense layers.
- Core Operations:
- Convolution:
- Stride, padding control output size:
- $H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{s} \right\rfloor + 1$

CNN (CONV2D + POOLING) CODE

All Rights Reserved. Author @ Rajendra Phani

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

RNNs, LSTMs, and GRUs are recurrent neural network architectures designed for processing sequential data like text or time series by maintaining a hidden state that captures information from previous inputs .

RNNs

Recurrent Neural Networks (RNNs) feature a looped structure where the hidden state updates at each time step using the current input and prior hidden state, enabling memory of past context . Benefits include efficient parameter sharing across time steps, reducing model size, and flexibility for variable-length sequences in tasks like speech recognition .

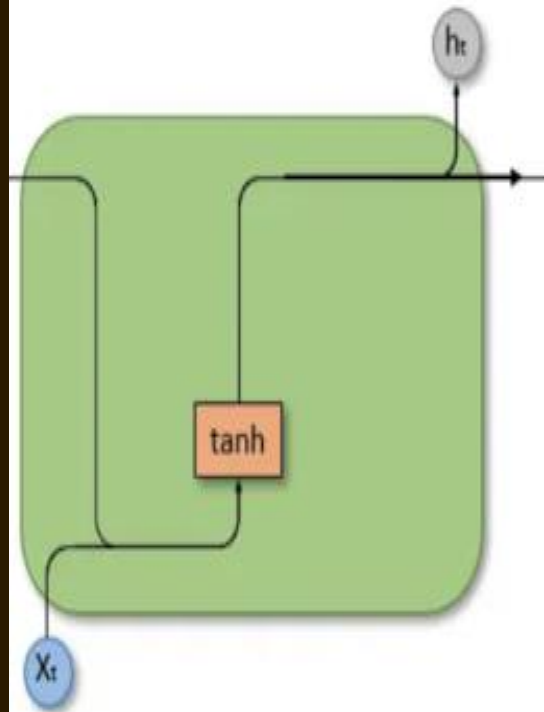
LSTMs

Long Short-Term Memory (LSTM) networks extend RNNs with memory cells and three gates—input, forget, and output—to regulate information flow and mitigate vanishing gradients . They excel at capturing long-term dependencies, making them ideal for time-series forecasting and text generation, though they require more parameters and training time .

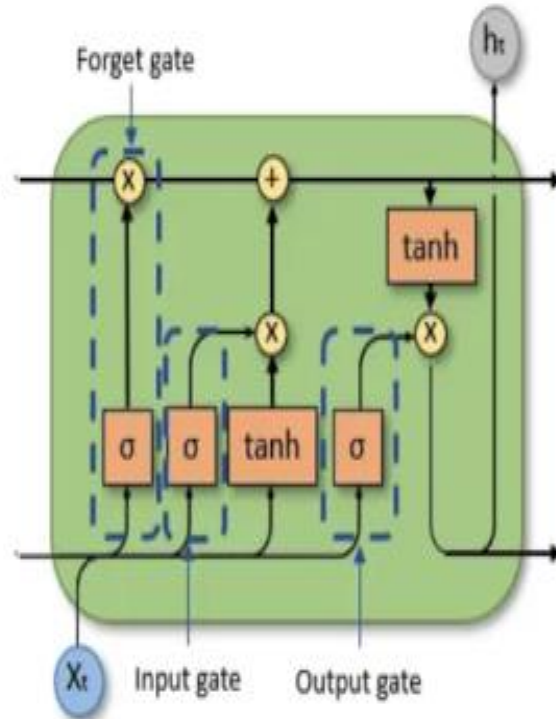
GRUs

Gated Recurrent Units (GRUs) simplify LSTMs by merging input and forget gates into an update gate plus a reset gate, using fewer parameters for faster training . They balance efficiency and performance on long sequences, suiting applications like NLP where computational speed matters over LSTM's complexity

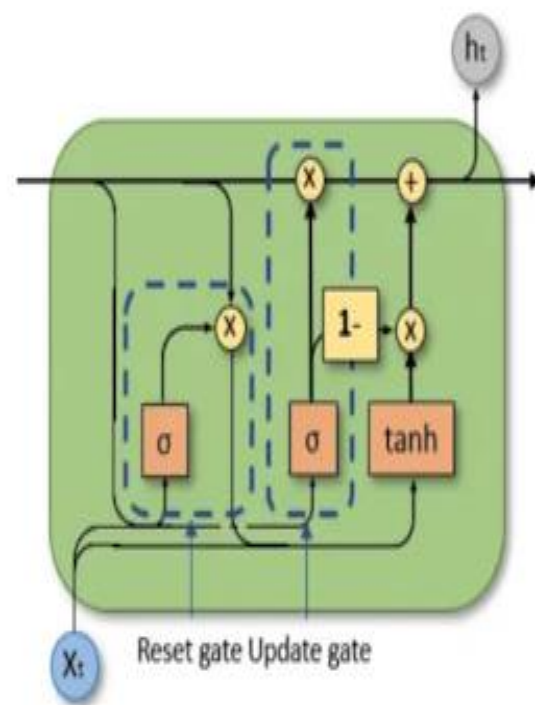
RNN



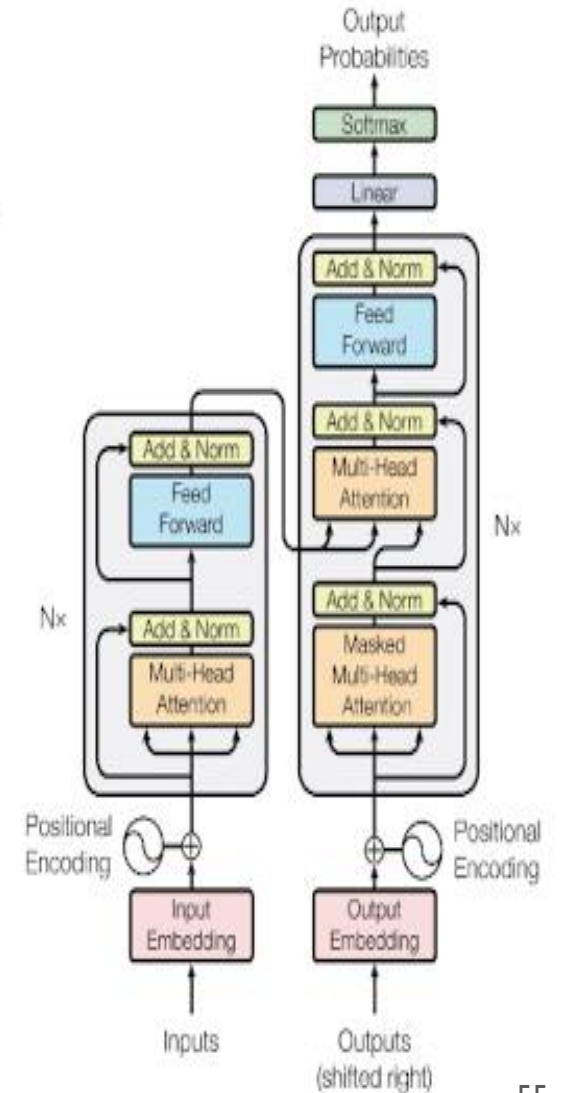
LSTM

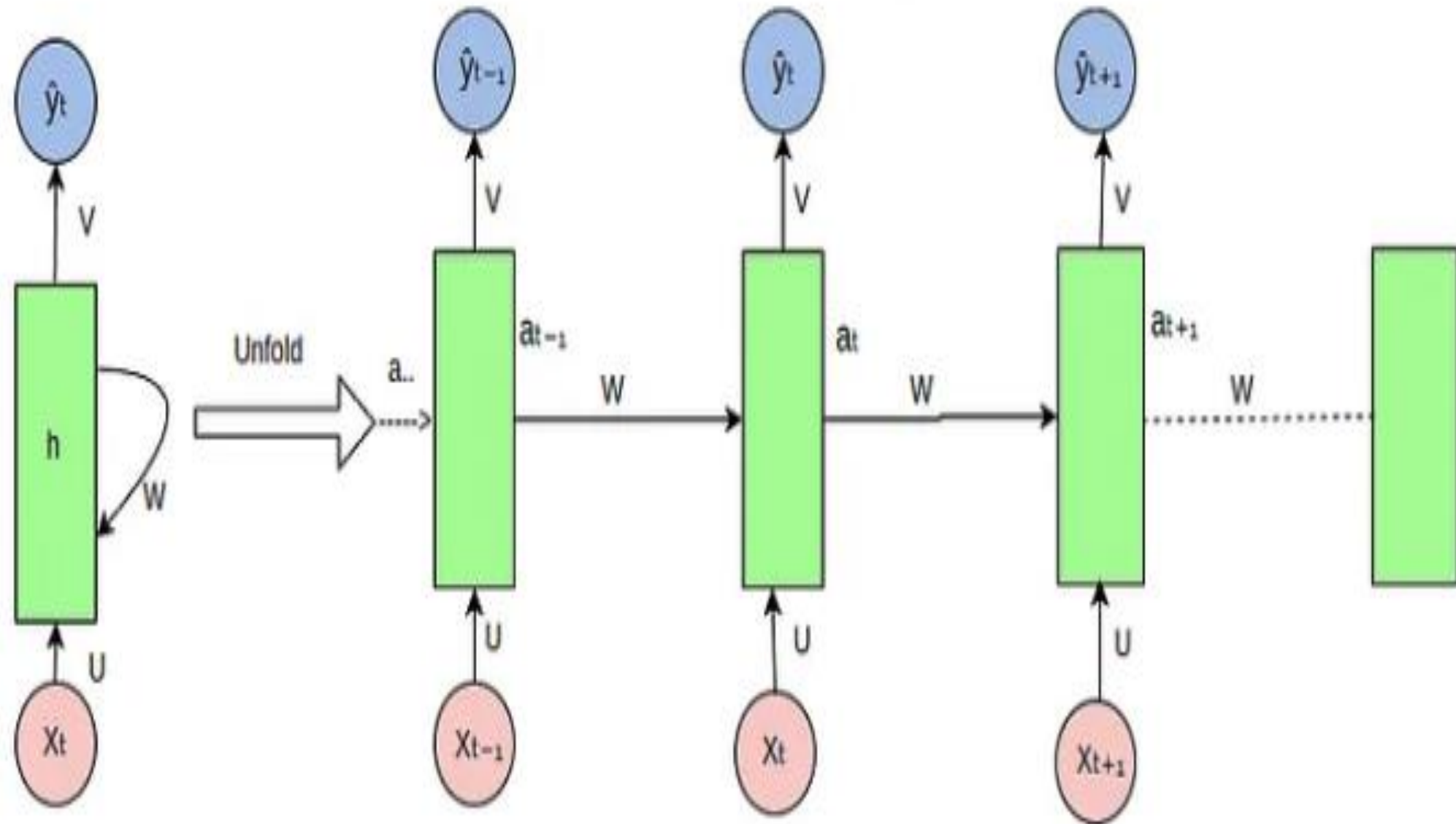


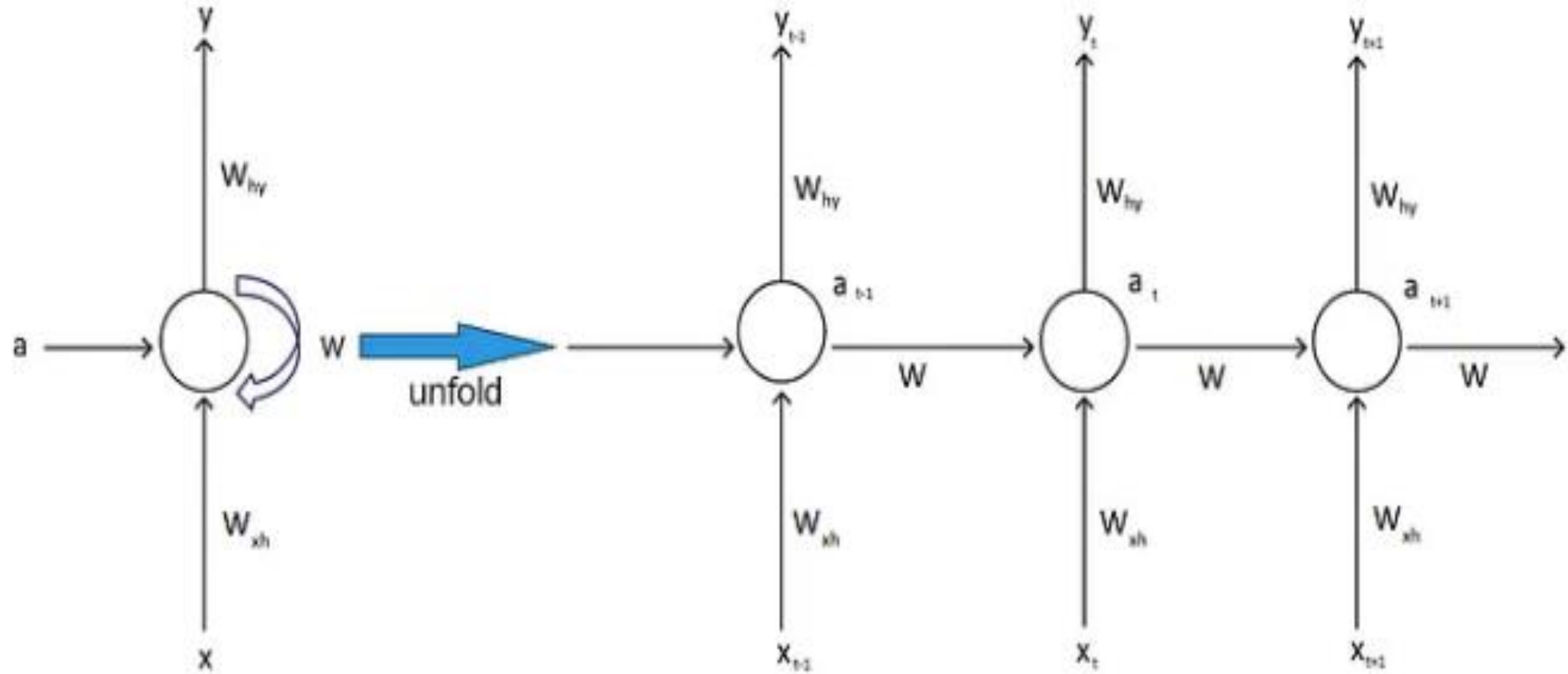
GRU



Transformers

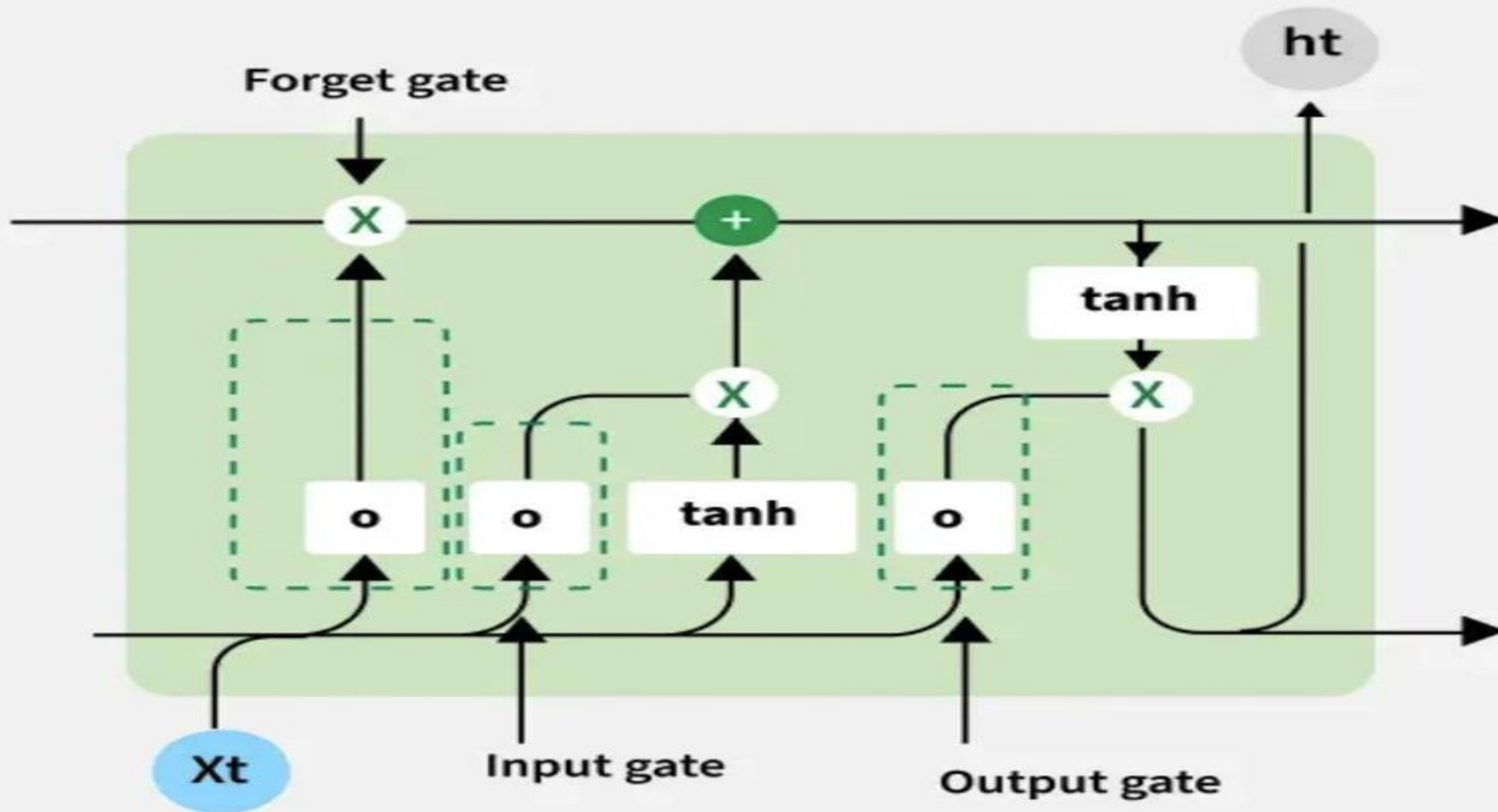




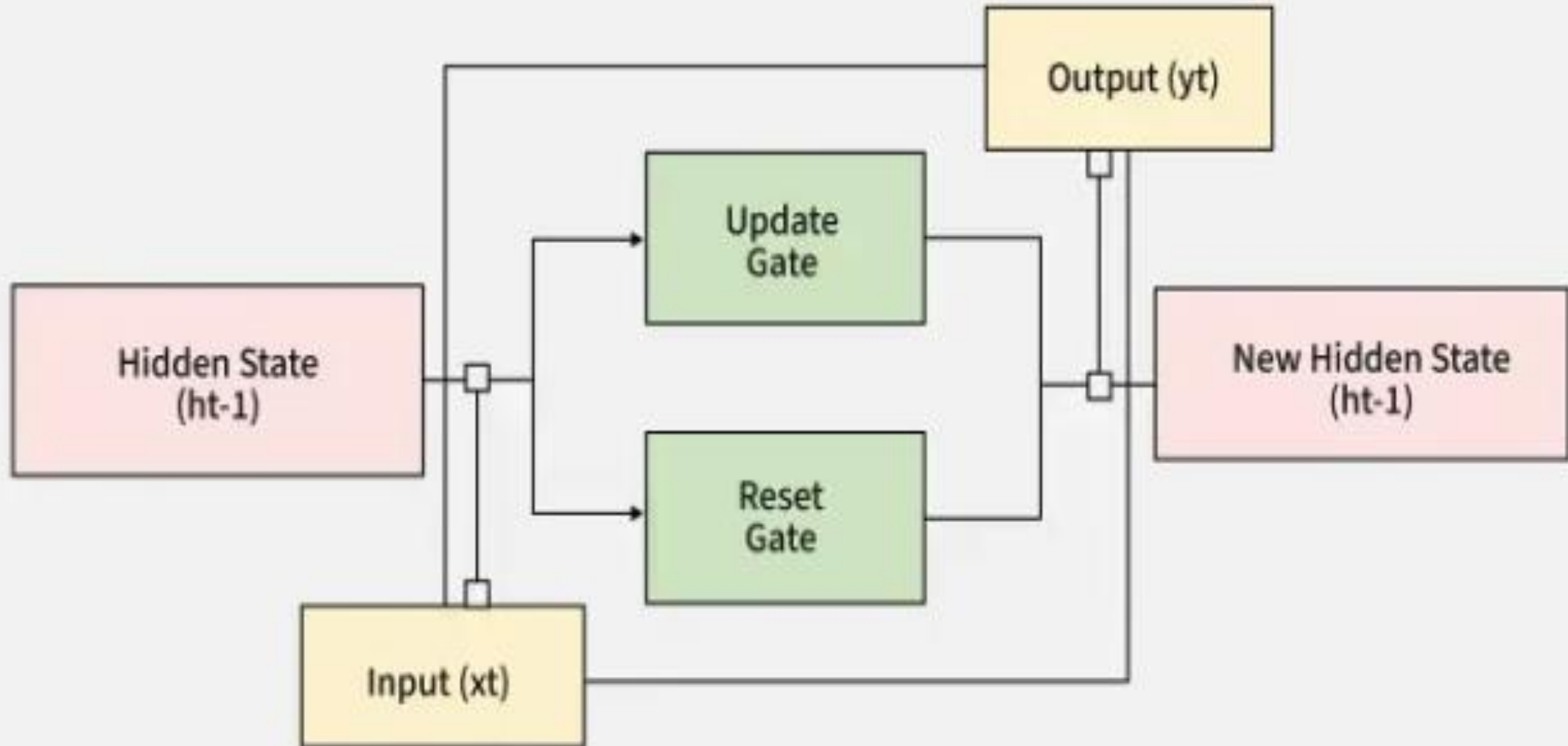


Recurrent Neural Networks (RNNs)

LSTM



How GRU Works



13. RNNS, LSTM, GRU

All Rights Reserved. Author @ Rajendra Phani

- RNN Cell:
- $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$
- Problem: vanishing/exploding gradients $\rightarrow$ can't learn long-term dependencies.
- LSTM (Long Short-Term Memory):
- Gates:
 - Forget: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
 - Input: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
 - Candidate:
 - Cell update: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
 - Output: $o_t = \sigma(\dots)$
- GRU (simpler): 2 gates (reset, update); no cell state.
- LSTM/GRU for sequences (e.g., sentiment, translation, time series).

RNN, LSTM, GRU CODE

All Rights Reserved. Author @ Rajendra Phani

```
# Simple RNN
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(64, return_sequences=True, input_shape=(None, 10)),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])
model.summary()

# LSTM (preferred)
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(64, return_sequences=True, input_shape=(None, 10)),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(1)
])
model.summary()

# GRU (faster, often similar perf)
model = tf.keras.Sequential([
    tf.keras.layers.GRU(64, input_shape=(None, 10)),
    tf.keras.layers.Dense(1)
])
model.summary()
```

Autoencoders are unsupervised neural networks that compress input data into a lower-dimensional latent representation and reconstruct it, enabling tasks like dimensionality reduction and denoising. Variational Autoencoders (VAEs) extend this by modeling the latent space as a probability distribution, typically Gaussian, to generate new data samples.

Autoencoder Architecture

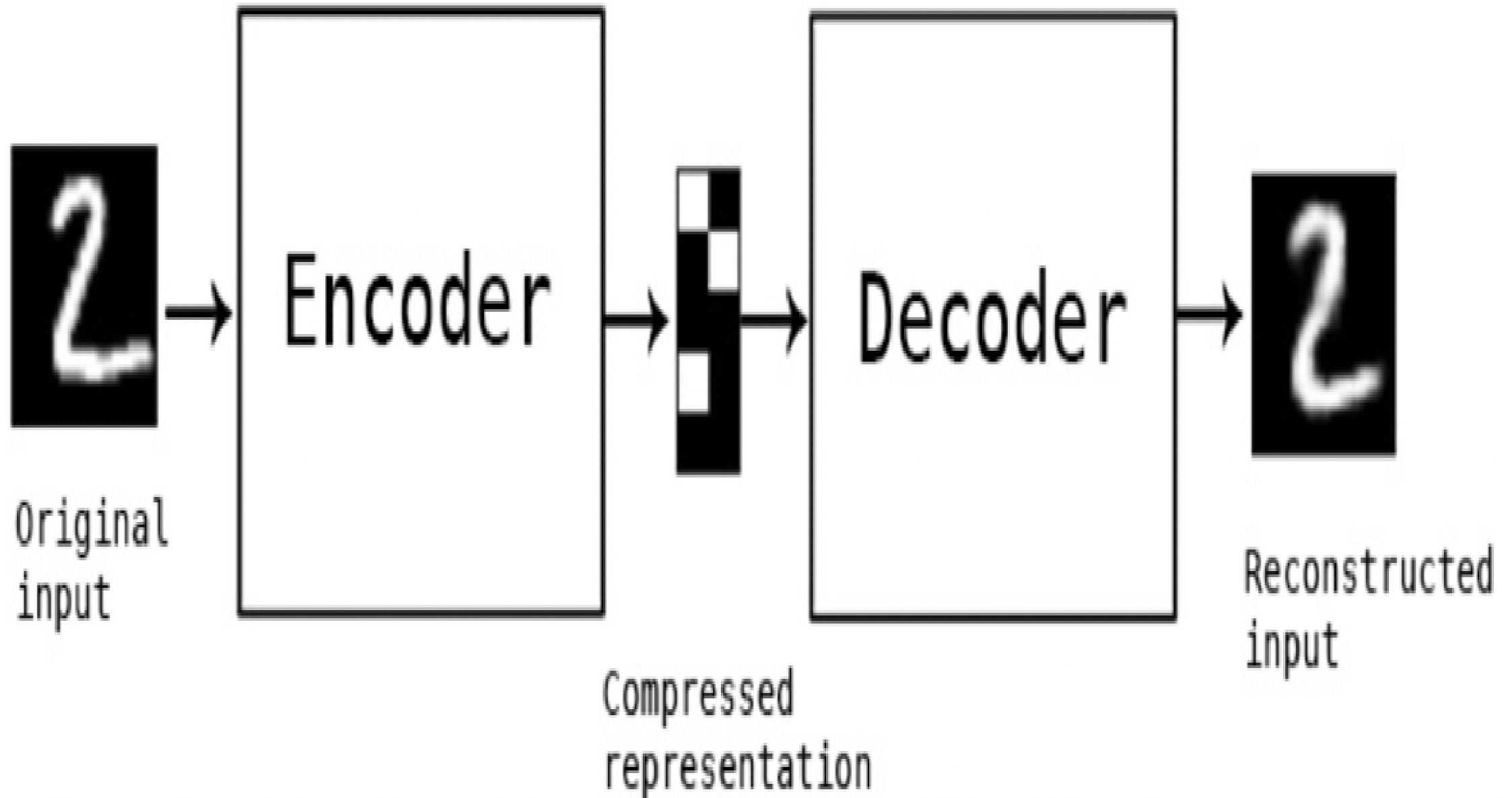
Standard autoencoders consist of an encoder, a latent code (bottleneck), and a decoder. The encoder maps high-dimensional input to a compact latent vector, while the decoder reconstructs the output to match the input, minimizing reconstruction loss.

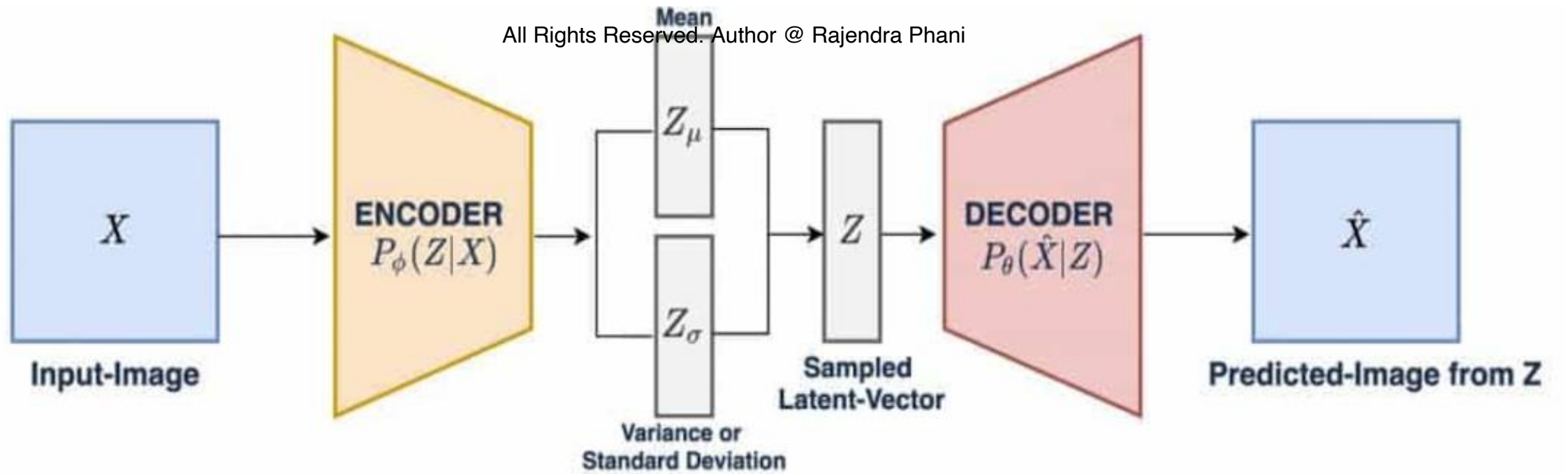
VAE Architecture

VAEs share the encoder-decoder structure but modify the encoder to output parameters (mean and variance) of a distribution from which latent samples are drawn via reparameterization. The decoder then generates from these samples, with training optimizing both reconstruction loss and a KL-divergence term to regularize the latent space toward a prior distribution.

Key Differences

- Autoencoders produce deterministic fixed-point encodings, suitable for compression but limited for generation.
- VAEs enable probabilistic sampling for continuous latent spaces, supporting data generation and interpolation. This makes VAEs powerful for generative tasks like image synthesis.

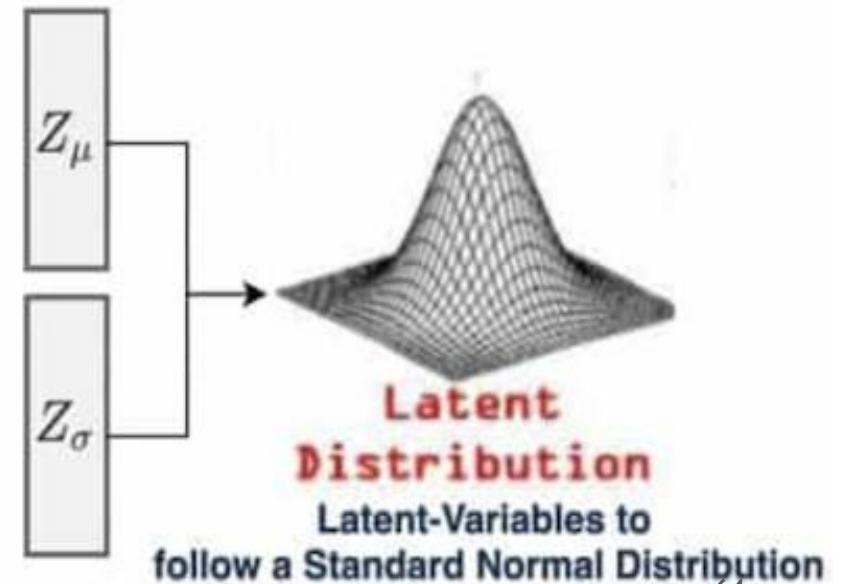




Sample a point from $G(Z_{\mu}, Z_{\sigma})$

$$Z = \mu + \sigma \odot \epsilon$$

$$\epsilon \sim \mathcal{N}(0, 1)$$



14. AUTOENCODERS & VAEs

All Rights Reserved. Author @ Rajendra Phani

- Autoencoder (AE):
- Encoder: $z = f(x)$
- Decoder: $\hat{x} = g(z)$
- Loss: $\|x - \hat{x}\|^2$
- Use: dimensionality reduction, denoising, anomaly detection.
- Variational Autoencoder (VAE):
- Probabilistic encoder: $q(z | x) = \mathcal{N}(\mu(x), \sigma(x))$
- Reparameterization trick:
- $z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(0, I)$
- Loss:
- $$\mathcal{L} = \underbrace{\mathbb{E}_{q(z|x)} [\log p(x | z)]}_{\text{recon}} - \underbrace{D_{KL}(q(z | x) \parallel p(z))}_{\text{regularizer}}$$
- Enables *generation* by sampling $z \sim p(z)$ e.g., $\mathcal{N}(0, I)$.

AUTOENCODER CODE

All Rights Reserved. Author @ Rajendra Phani

```
# Encoder
```

```
input_img = tf.keras.Input(shape=(784,))
```

```
encoded = tf.keras.layers.Dense(32, activation='relu')(input_img)
```

```
# Decoder
```

```
decoded = tf.keras.layers.Dense(784, activation='sigmoid')(encoded)
```

```
autoencoder = tf.keras.Model(input_img, decoded)
```

```
encoder = tf.keras.Model(input_img, encoded)
```

```
autoencoder.compile(optimizer='adam', loss='mse')
```

```
# Train on noisy → clean images
```

VAE (MINIMAL WORKING)

All Rights Reserved. Author @ Rajendra Phani

```
class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.random.normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

# Encoder
inputs = tf.keras.Input(shape=(784,))
x = tf.keras.layers.Dense(64, activation='relu')(inputs)
z_mean = tf.keras.layers.Dense(2)(x)
z_log_var = tf.keras.layers.Dense(2)(x)
z = Sampling()(z_mean, z_log_var)

# Decoder
decoder_input = tf.keras.Input(shape=(2,))
x = tf.keras.layers.Dense(64, activation='relu')(decoder_input)
outputs = tf.keras.layers.Dense(784, activation='sigmoid')(x)
decoder = tf.keras.Model(decoder_input, outputs)

# VAE model
outputs = decoder(z)
vae = tf.keras.Model(inputs, outputs)

# Custom loss (reconstruction + KL)
reconstruction_loss = tf.reduce_mean(tf.keras.losses.binary_crossentropy(inputs, outputs)) * 784
kl_loss = -0.5 * tf.reduce_mean(tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1))
vae.add_loss(reconstruction_loss + kl_loss)

vae.compile(optimizer='adam')
```

All Rights Reserved. Author @ Rajendra Phani

GENERATIVE ADVERSARIAL NETWORKS (GANs)

All Rights Reserved. Author @ Rajendra Phani

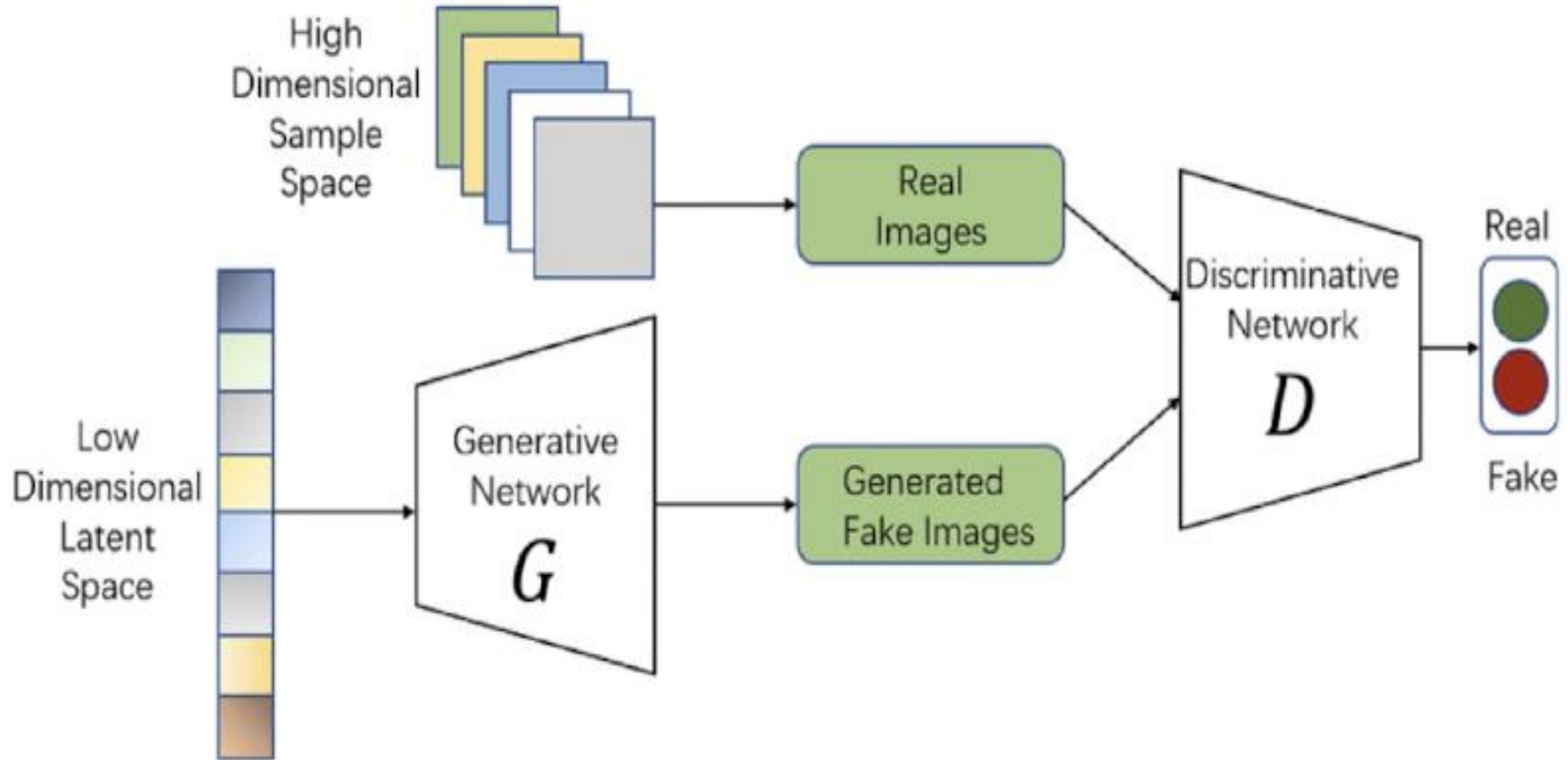
Generative Adversarial Networks (GANs) are a deep learning framework where two neural networks—a generator and a discriminator—compete adversarially to produce realistic synthetic data, such as images or text, by learning patterns from real datasets.

Core Components

The generator takes random noise as input and transforms it through layers like dense or convolutional networks to create fake data samples resembling real ones. The discriminator, often a binary classifier using architectures like convolutional neural networks (CNNs), evaluates whether inputs are real (from the true data distribution) or fake (from the generator), outputting a probability score.

Training Process

Training alternates between updating the discriminator on real and generated data to improve its classification accuracy, and updating the generator to maximize the discriminator's errors via backpropagation. This zero-sum game continues until equilibrium, where the discriminator cannot reliably distinguish real from fake, indicating the generator produces high-quality outputs. Loss functions, such as those minimizing the divergence between real and generated distributions, drive this adversarial improvement.



15. GENERATIVE ADVERSARIAL NETWORKS (GANS)

All Rights Reserved. Author @ Rajendra Phani

- Two networks compete:
- Generator $G: z \sim p_z \rightarrow x_{fake} = G(z)$
- Discriminator D : outputs $P(real | x)$
- Minimax Objective:
- Training (alternate steps):
- Fix G , train D to distinguish real/fake
- Fix D , train G to fool D
- Challenges: mode collapse, instability
Improvements: DCGAN, WGAN, StyleGAN

GAN (SIMPLE MNIST) CODE

All Rights Reserved. Author @ Rajendra Phani

```
# Generator: z (100,) → image (28,28,1)
generator = tf.keras.Sequential([
    tf.keras.layers.Dense(256, input_dim=100, activation='relu'),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(784, activation='tanh'),
    tf.keras.layers.Reshape((28,28,1))
])

# Discriminator: image → real/fake prob
discriminator = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28,1)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# GAN = generator + discriminator (frozen)
discriminator.trainable = False
gan_input = tf.keras.Input(shape=(100,))
gan_output = discriminator(generator(gan_input))
gan = tf.keras.Model(gan_input, gan_output)

# Compile
generator.compile(optimizer='adam', loss='binary_crossentropy')
discriminator.compile(optimizer='adam', loss='binary_crossentropy')
gan.compile(optimizer='adam', loss='binary_crossentropy')
```

All Rights Reserved. Author @ Rajendra Phani

Attention is a mechanism in deep learning that allows models to focus on the most relevant parts of an input sequence when making predictions. In transformers, attention is mainly implemented as self-attention, where each token in a sequence attends to, or relates to, other tokens in the same sequence. This helps the model capture long-range dependencies and contextual relationships, meaning it can understand the meaning of words based on the entire sentence or sequence context rather than just locally adjacent tokens.

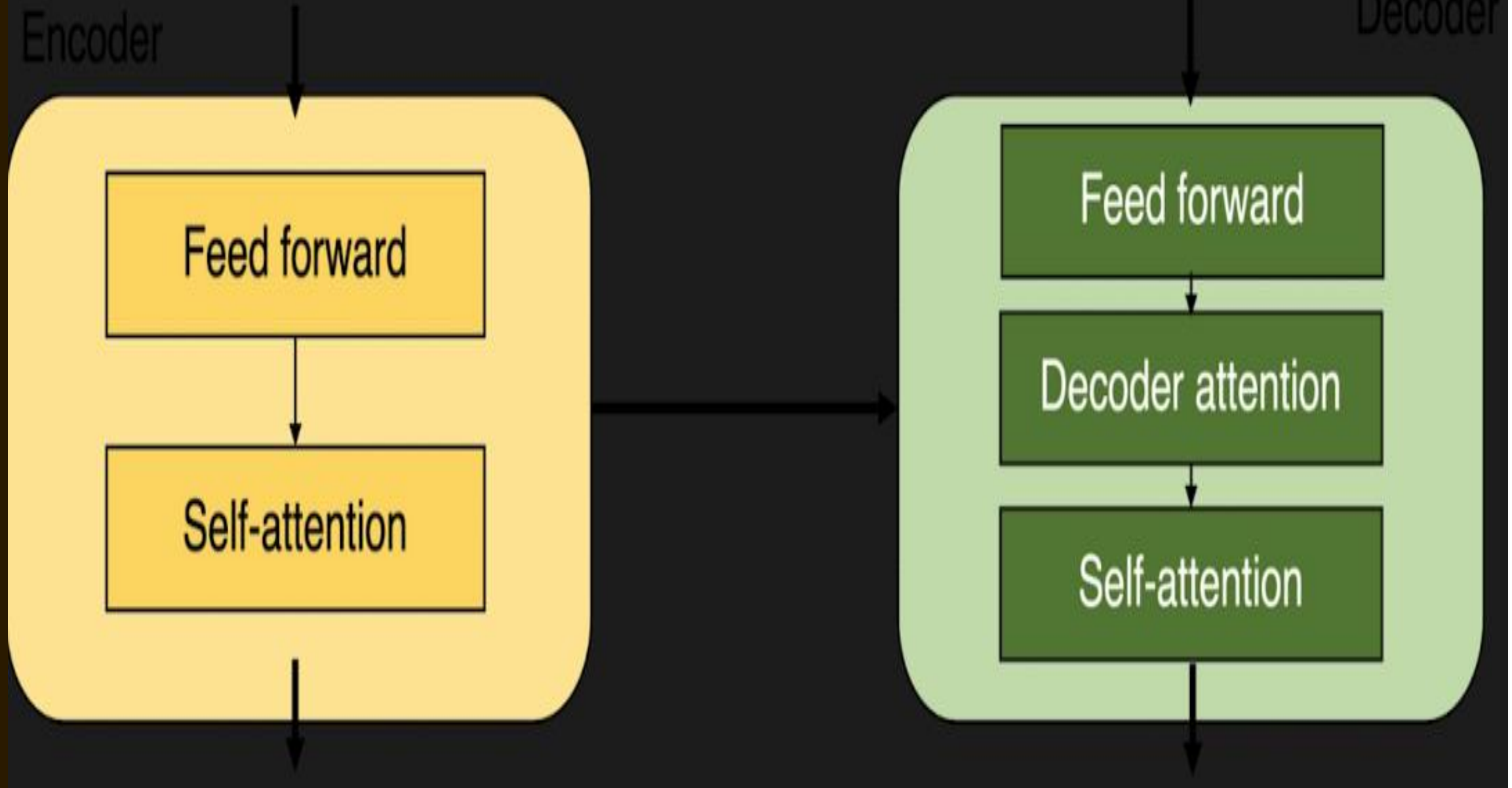
Transformers are a neural network architecture that leverages this multi-head self-attention mechanism along with feedforward neural networks, arranged in layers, to process sequences of data efficiently and in parallel. Unlike traditional recurrent neural networks (RNNs) which process sequential data step-by-step, transformers process the entire input sequence simultaneously, which enables faster training and better handling of long-range interactions in data. The model architecture typically consists of an encoder and a decoder. The encoder creates contextual token representations by combining information across tokens via self-attention. The decoder generates output sequences using both self-attention and cross-attention mechanisms, allowing it to focus on relevant parts of the input sequence and previously generated tokens.

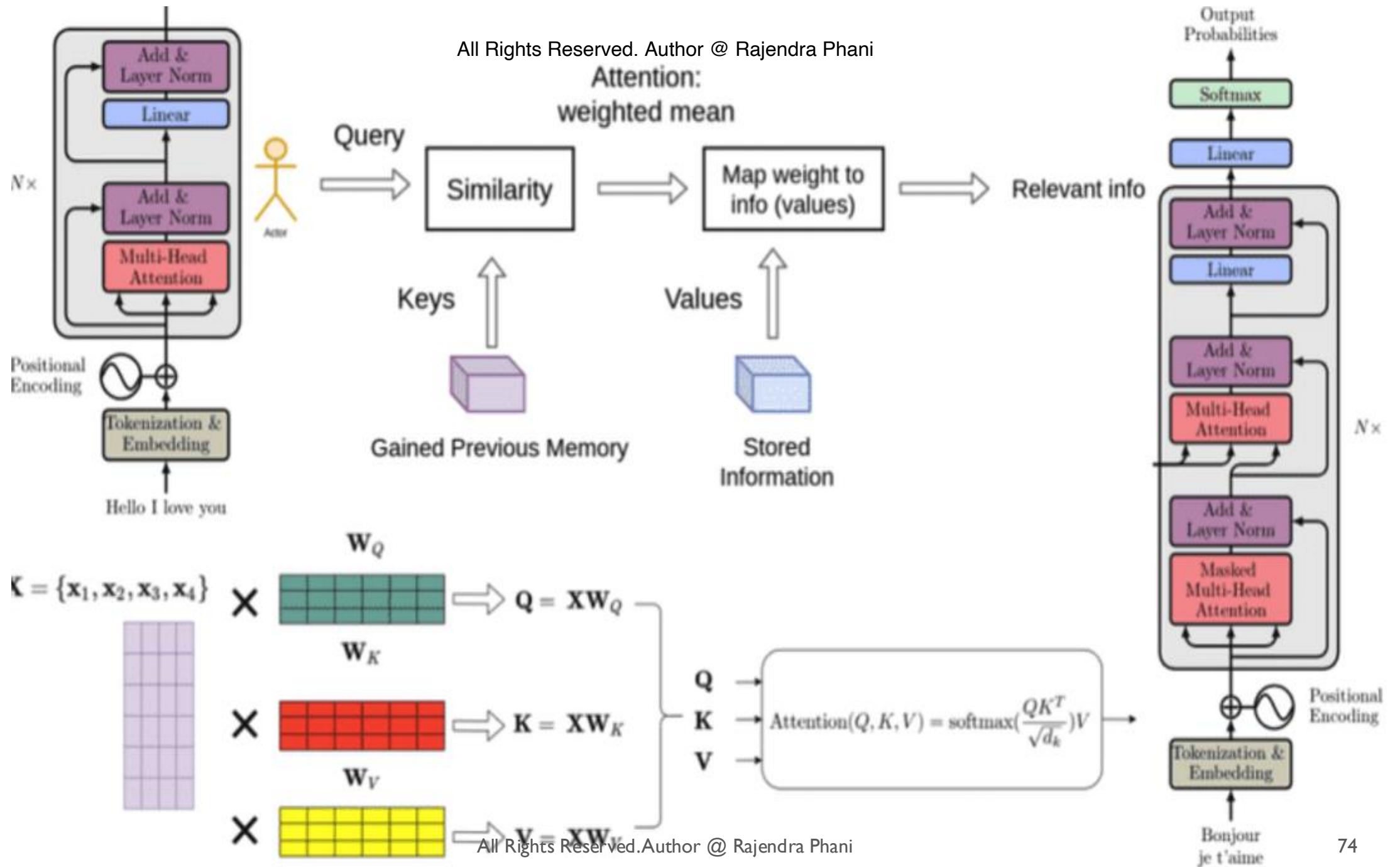
Multi-head attention is an enhanced attention mechanism in transformers where multiple attention “heads” operate in parallel, each learning to focus on different aspects or relationships within the data. These heads’ outputs are concatenated and transformed to produce the final attention output, enriching the model’s ability to capture complex dependencies. Positional encoding is added to token embeddings to maintain the order of tokens, since the attention mechanism itself is order-agnostic.

This combination of self-attention, multi-head attention, feedforward layers, and positional encodings makes transformers highly effective for natural language processing tasks like language translation, text generation, and more recently, for image and speech processing. Transformers have enabled the development of large language models (LLMs) such as GPT, which process and generate human-like text by understanding context deeply within large datasets.

Encoder

Decoder





16. ATTENTION & TRANSFORMERS

All Rights Reserved. Author @ Rajendra Phani

- Scaled Dot-Product Attention:
- $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- $Q = XW^Q, K = XW^K, V = XW^V$
- Multi-Head Attention:
- $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$ $\text{head}_i = \text{Attention}(Q, W_i^Q, KW_i^K VW_i^V)$
- Transformer Block:
- Multi-Head Attention
- Add & Norm
- Feed-Forward (2 dense layers + ReLU)
- Add & Norm

MINIMAL TRANSFORMER BLOCK

All Rights Reserved. Author @ Rajendra Phani

```
from tensorflow.keras import layers

def transformer_block(x, d_model, num_heads, dff):
    # Multi-Head Attention
    attn_output = layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=d_model // num_heads
    )(x, x)
    x = layers.LayerNormalization()(x + attn_output)

    # Feed-Forward
    ffn = tf.keras.Sequential([
        layers.Dense(dff, activation='relu'),
        layers.Dense(d_model)
    ])
    ffn_output = ffn(x)
    return layers.LayerNormalization()(x + ffn_output)

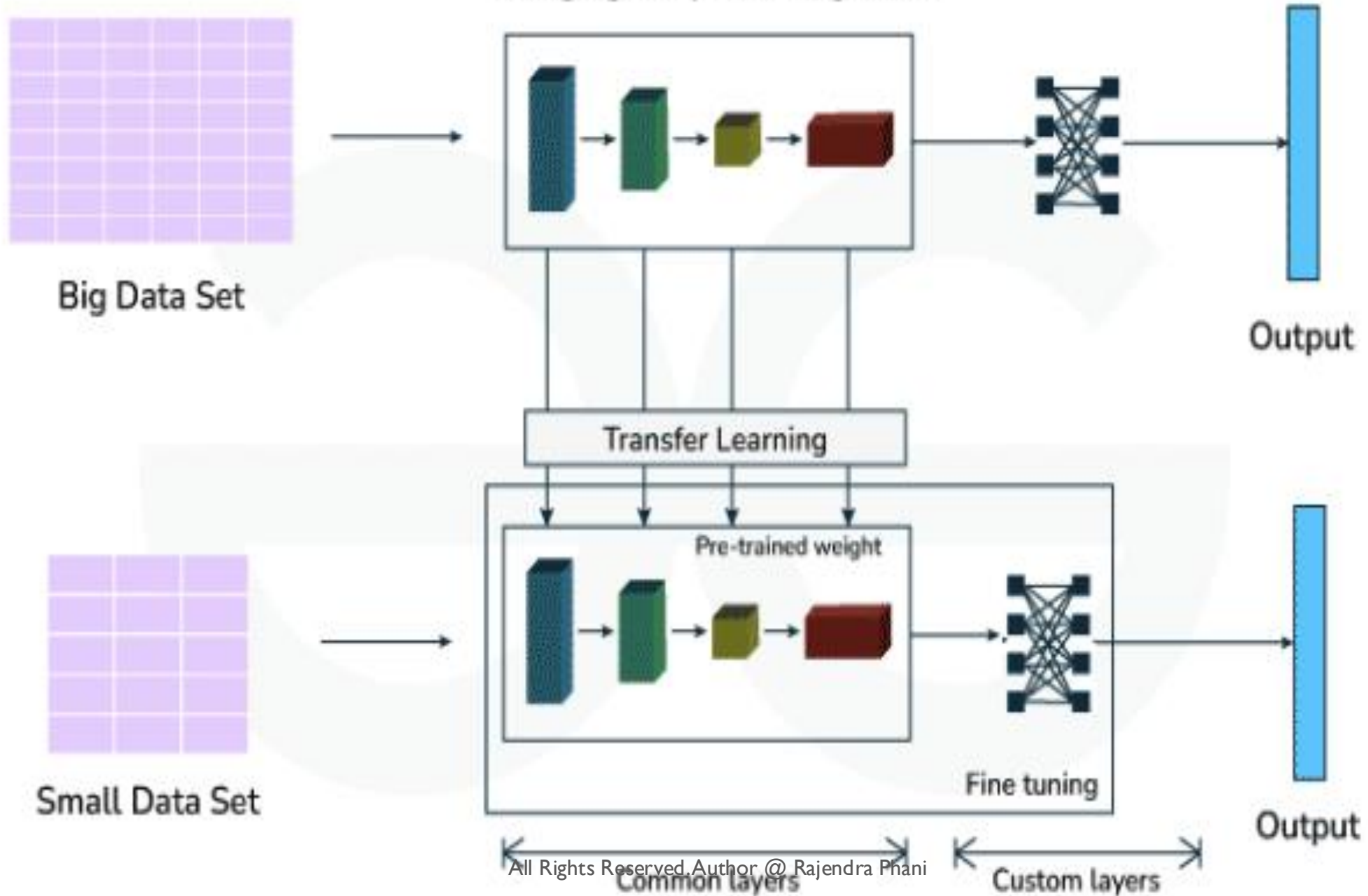
# Usage
x = tf.random.normal((2, 10, 64)) # (batch, seq_len, d_model)
out = transformer_block(x, d_model=64, num_heads=4, dff=256)
```

TRANSFER LEARNING & FINE-TUNING

All Rights Reserved. Author @ Rajendra Phani

Transfer learning is the technique of using a model that has already been trained on a large, general dataset and applying it to a new but related task by reusing the learned features. Typically, transfer learning involves freezing most of the pre-trained model's layers and retraining only the final layers to adapt to the new task.

Fine-tuning is a specific process within transfer learning where, after initially training new layers added for the new task, some or all of the pre-trained layers are “unfrozen” and retrained on the new dataset. This allows the model to slightly adjust its previously learned features to better fit the new task's specifics, thus improving performance. It balances retaining the model's general knowledge while specializing for the particular use case.



17. TRANSFER LEARNING & FINE-TUNING

All Rights Reserved. Author @ Rajendra Phani

- Idea: Reuse weights from pre-trained model (e.g., ResNet, BERT) on new task.
- Approaches:
- Feature extraction: Freeze base network; train only classifier head.
- Fine-tuning: Unfreeze some top layers + train with low LR.
- When to use?
- Small dataset → feature extraction
- Larger dataset → fine-tune

TRANSFER LEARNING (IMAGENET → CUSTOM)

All Rights Reserved. Author @ Rajendra Phani

```
# Load base model (no top classifier)
base_model = tf.keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,    # ← exclude final Dense(1000)
    input_shape=(224,224,3)
)

# Freeze base
base_model.trainable = False

# Add custom head
model = tf.keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax') # 10 new classes
])

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Later: fine-tune top layers
base_model.trainable = True
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), # lower LR!
              loss='sparse_categorical_crossentropy')
```